Laboratório de desenvolvimento de software

Maven

Gerenciamento de dependências

- Em um projeto de software, as dependências referem-se às bibliotecas, módulos ou componentes externos que são utilizados por sua aplicação para realizar determinadas funcionalidades.

- Em Java, essas dependências geralmente são gerenciadas por meio de ferramentas de gerenciamento de dependências, como o Maven, Gradle ou Ivy.

Maven

- É uma ferramenta para gerenciamento de projetos Java
  - Gerenciamento de dependências e estruturação do projeto;
- Objetivo: simplificar a criação e a distribuição de projetos;
- Recurso archetype: permite criar, automaticamente, a estrutura de um projeto;
  - Exemplos:
    - maven-archetype-webapp: cria a estrutura de uma aplicação web básica;
    - maven-archetype-j2ee-simple: cria a estrutura de uma aplicação Java EE

Maven

- Ele simplifica o processo de construção, relatórios e gestão de dependências, proporcionando uma estrutura coerente para o desenvolvimento de software.

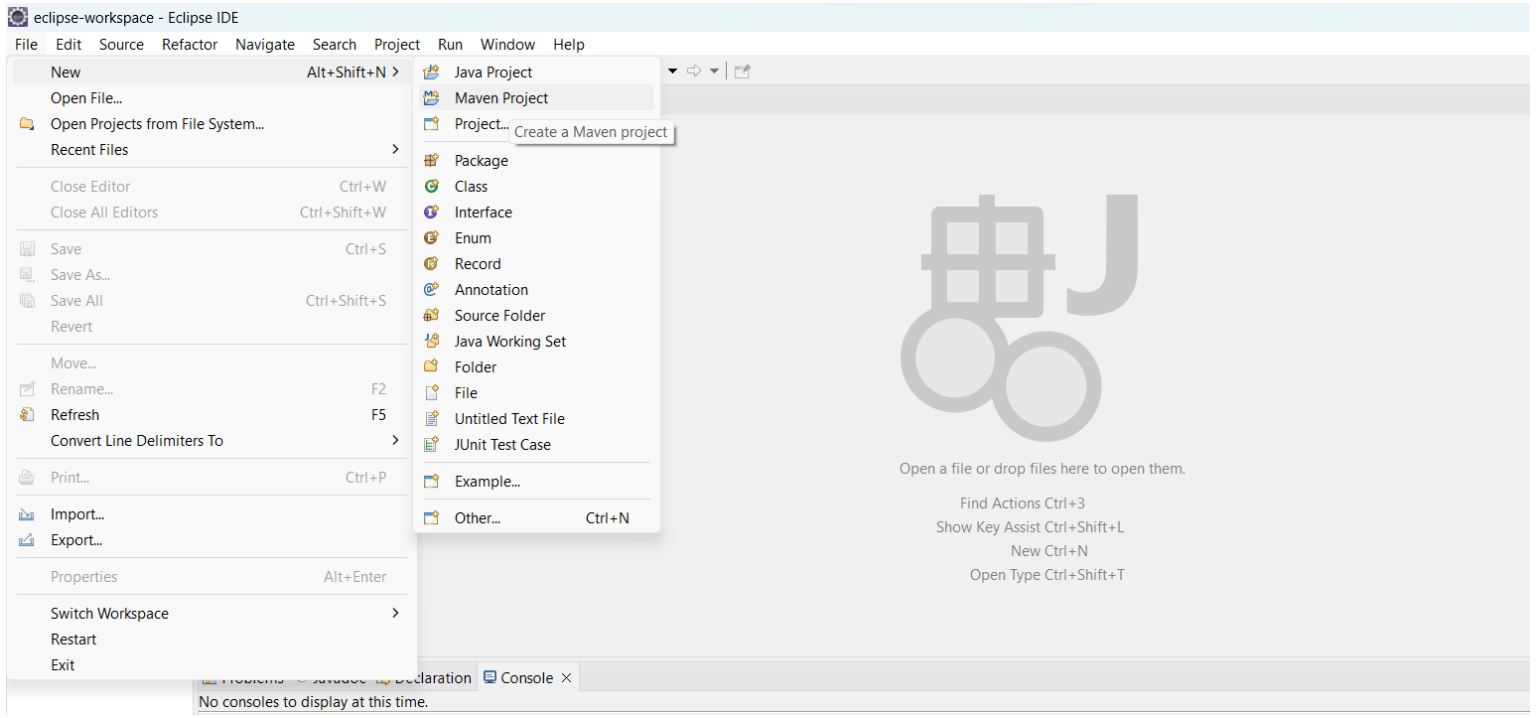
- Maven segue a filosofia de "convenção sobre configuração", o que significa que muitas configurações são definidas por padrão, reduzindo a complexidade para o desenvolvedor

Maven

- Project Object Model (POM)
  - O POM, ou Modelo de Objeto de Projeto, é um conceito fundamental no Maven.
  - É um arquivo XML que descreve o projeto, suas configurações e suas dependências.
  - O POM contém informações como nome do projeto, versão, recursos, plugins, dependências e outros detalhes de configuração.

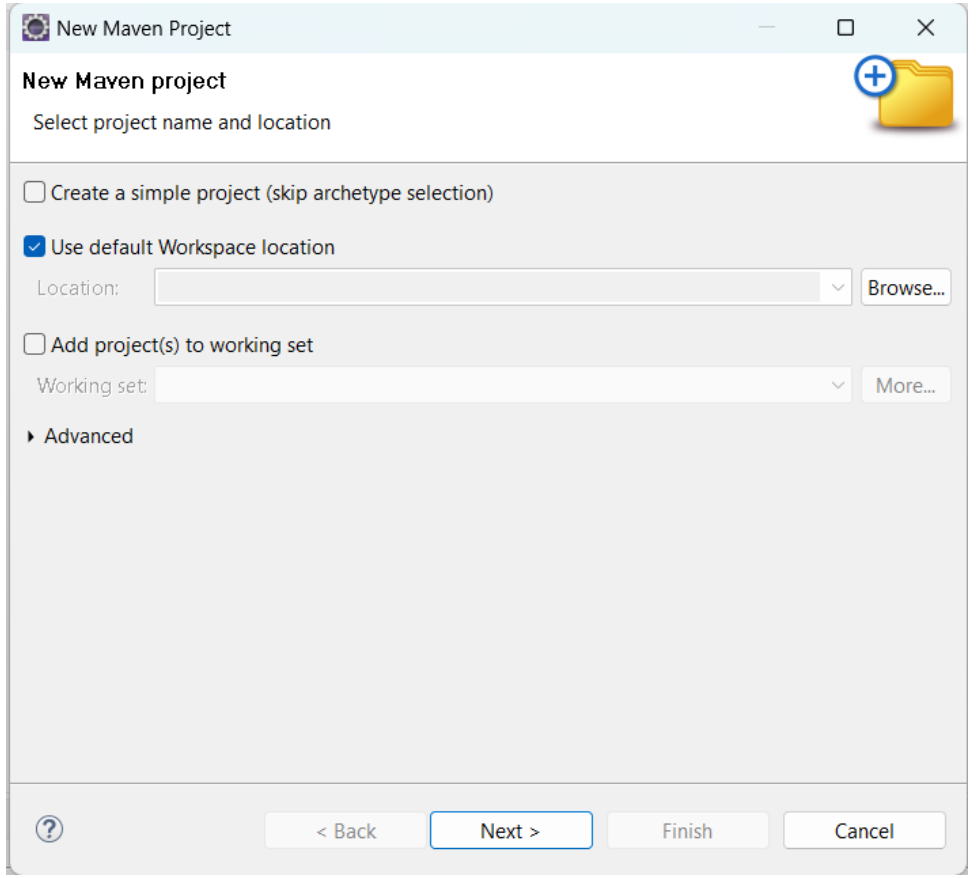
Maven

- Agora, vamos voltar a utilizar o Eclipse.
  - Abra o eclipse e clique em File -> New -> Maven Project



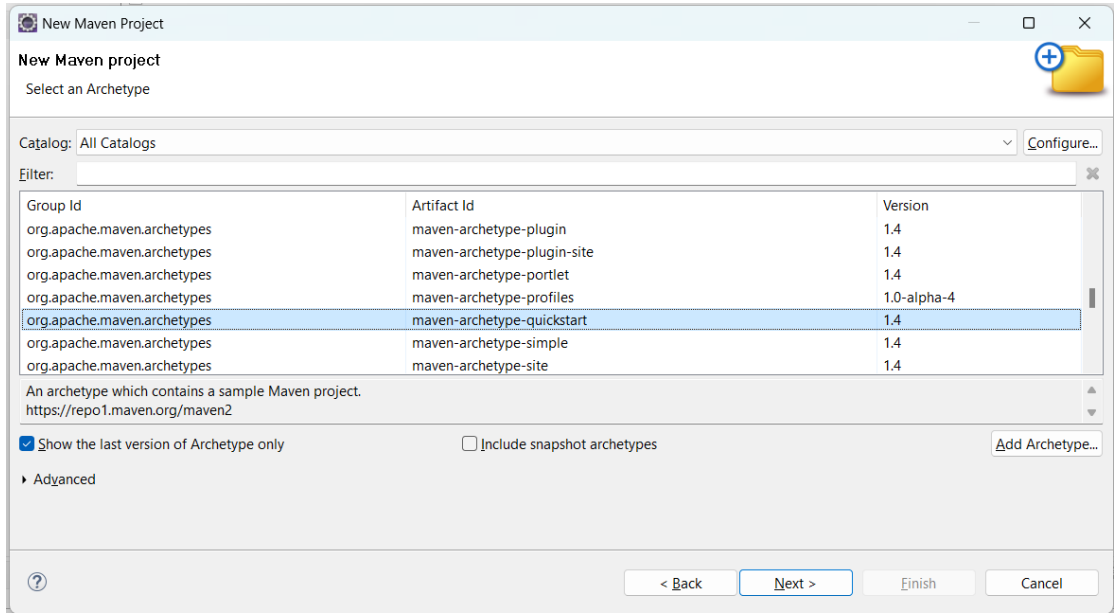
Maven

- Após clique em Next



Maven

- Archetypes: são modelos de projetos prontos com o Maven configurado

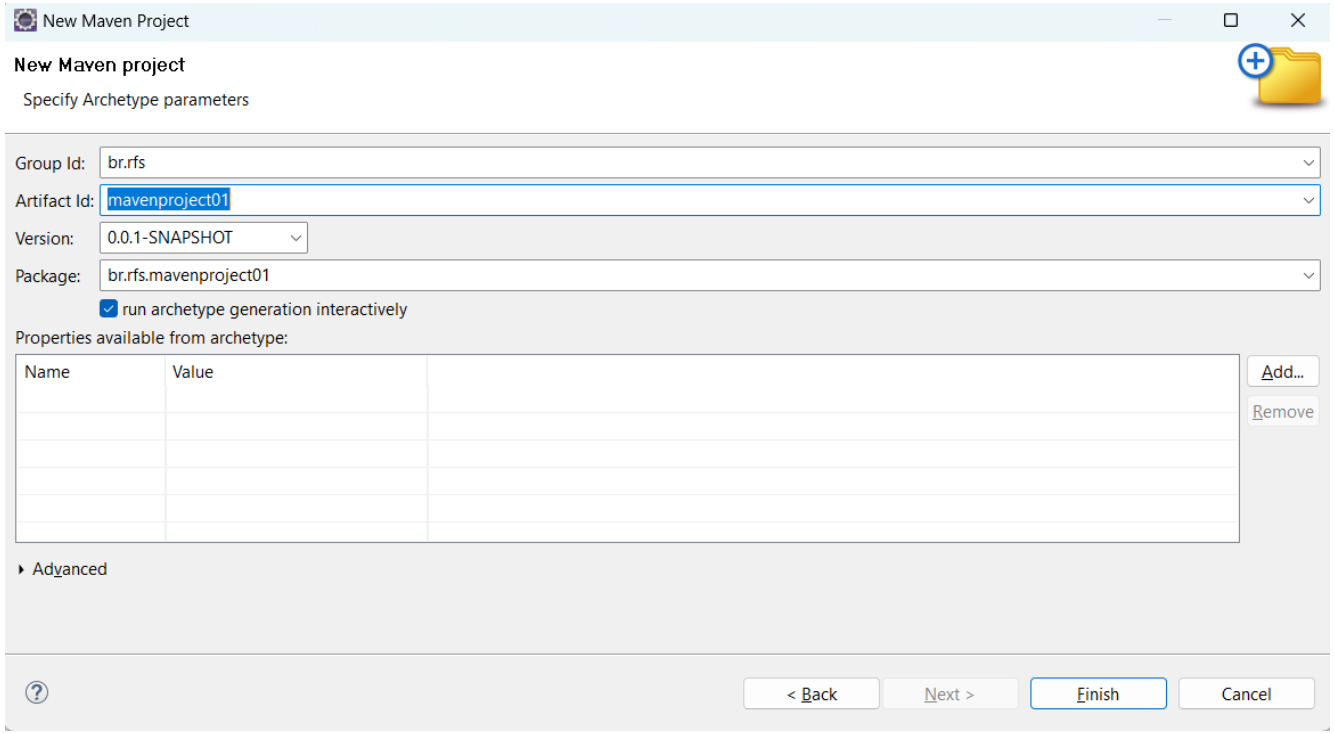


- Aguardar um tempo o download dos modelos

- Selecionar maven-archetype-quickstart

Maven

- Identificadores obrigatórios: adicionados ao pom.xml

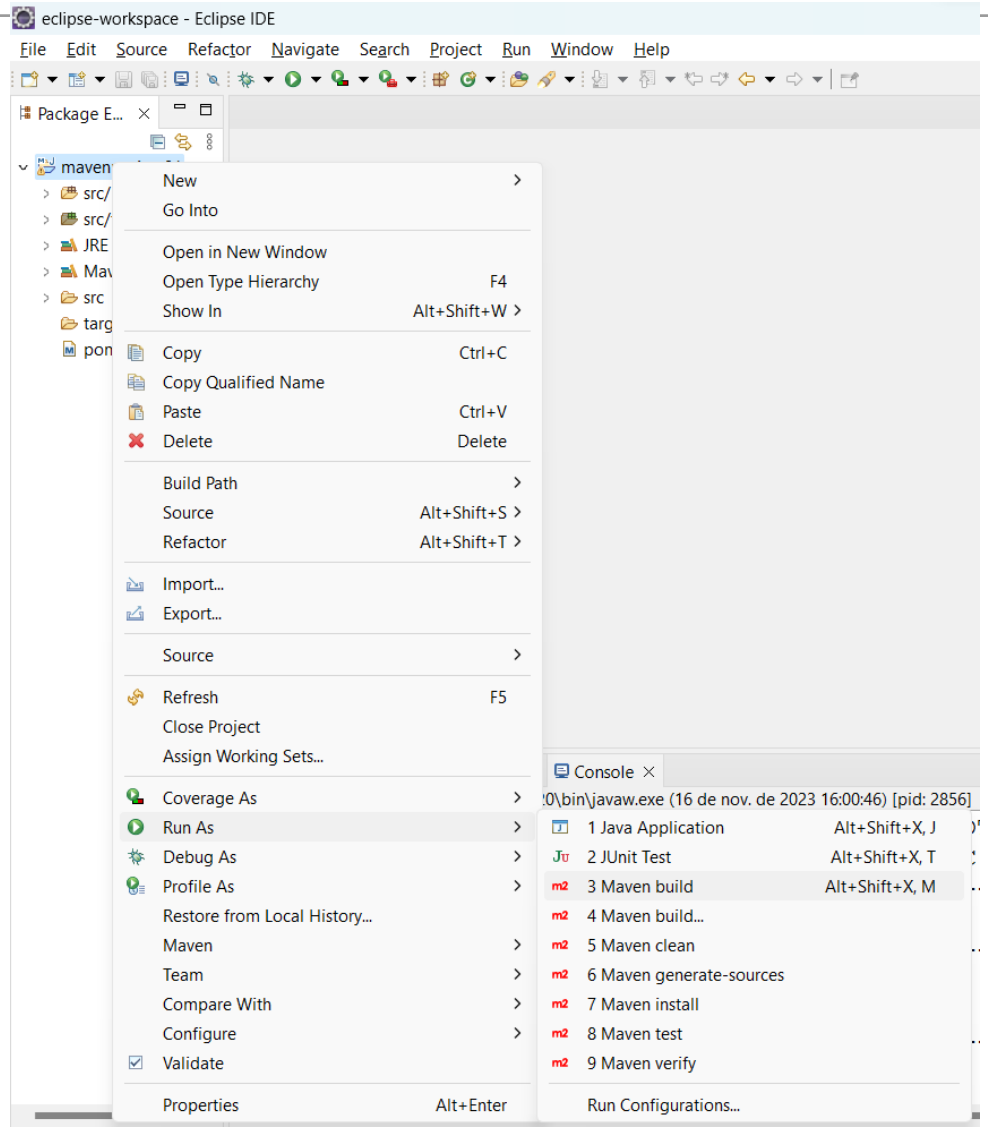


- prefixo dos pacotes do projeto
- nome do projeto criado
- Versão do projeto: padrão
- nome do pacote (Group Id + Artifact Id)

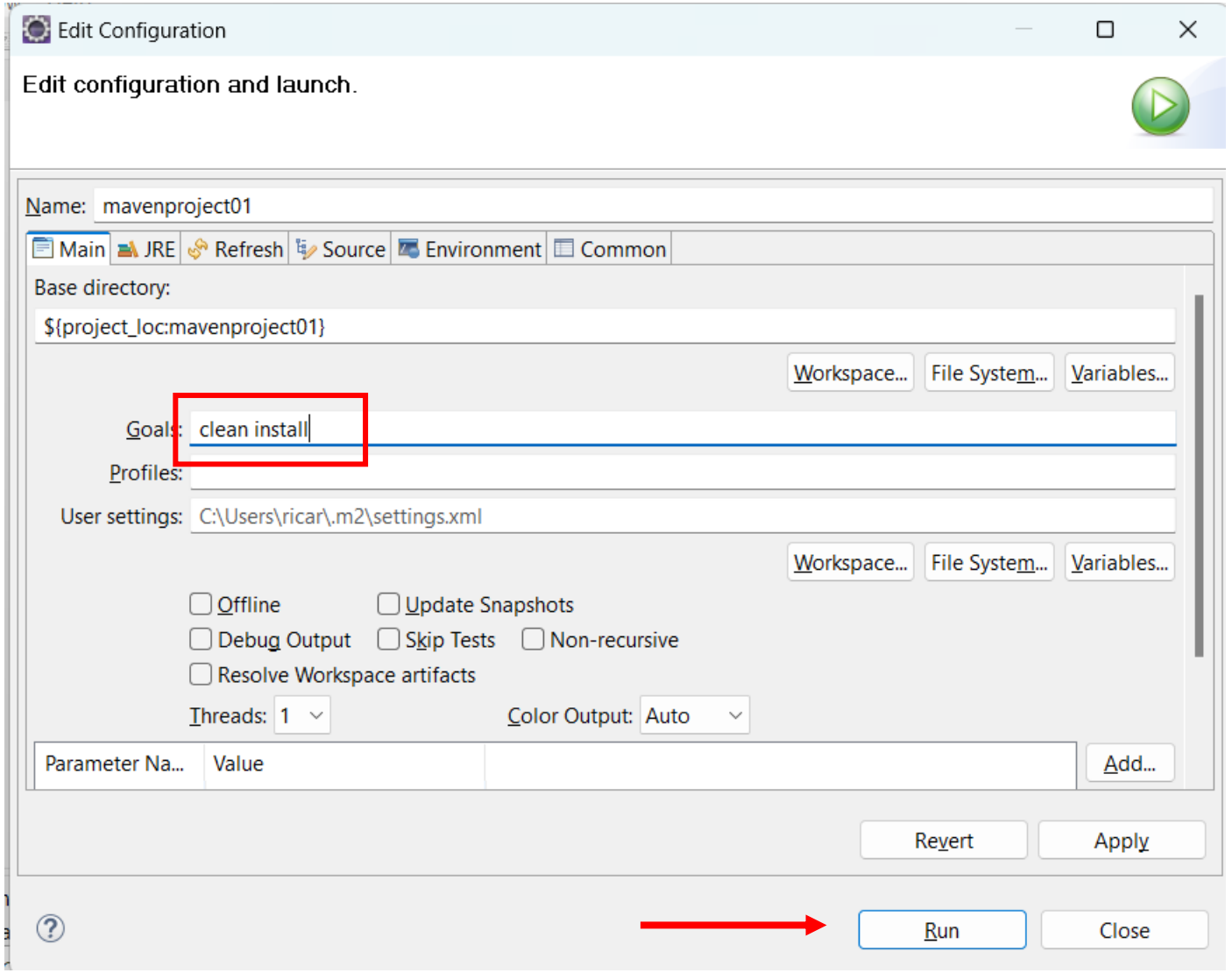
Maven

- Será necessário confirmar as configurações no console e após, o projeto será montado.
  - Aperte Y e aperte ENTER!
- Após, poderemos compilar!

Maven


eclipse-workspace - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package E...
New
Go Into
Open in New Window
Open Type Hierarchy F4
Show In Alt+Shift+W
Copy Ctrl+C
Copy Qualified Name
Paste Ctrl+V
Delete Delete
Build Path
Source Alt+Shift+S
Refactor Alt+Shift+T
Import...
Export...
Source
Refresh F5
Close Project
Assign Working Sets...
Coverage As
Run As
Debug As
Profile As
Restore from Local History...
Maven
Team
Compare With
Configure
Validate
Properties Alt+Enter
Console
1 Java Application Alt+Shift+X, J
2 JUnit Test Alt+Shift+X, T
3 Maven build Alt+Shift+X, M
4 Maven build...
5 Maven clean
6 Maven generate-sources
7 Maven install
8 Maven test
9 Maven verify
Run Configurations...

Maven


Edit Configuration
Edit configuration and launch.
Name: mavenproject01
Main
JRE
Refresh
Source
Environment
Common
Base directory:
${project_loc:mavenproject01}
Workspace...
File System...
Variables...
Goals: clean install
Profiles:
User settings: C:\Users\ricar\.m2\settings.xml
Workspace...
File System...
Variables...
Offline
Update Snapshots
Debug Output
Skip Tests
Non-recursive
Resolve Workspace artifacts
Threads: 1
Color Output: Auto
Parameter Na...
Value
Add...
Revert
Apply
Run
Close

Maven

- Se ocorrer este erro:

```
[ERROR] COMPILATION ERROR :
[INFO] -------------------------------------------------------------
[ERROR] Source option 7 is no longer supported. Use 8 or later.
[ERROR] Target option 7 is no longer supported. Use 8 or later.
[INFO] 2 errors
[INFO] -------------------------------------------------------------
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.753 s
[INFO] Finished at: 2023-11-16T16:12:49-03:00
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.8.0:compile (default-compile) on pr
[ERROR] Source option 7 is no longer supported. Use 8 or later.
[ERROR] Target option 7 is no longer supported. Use 8 or later.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
```

Maven

- No seu projeto Maven, abra o arquivo pom.xml e verifique se a configuração do compilador Java (maven-compiler-plugin) está definida corretamente para uma versão do Java suportada.

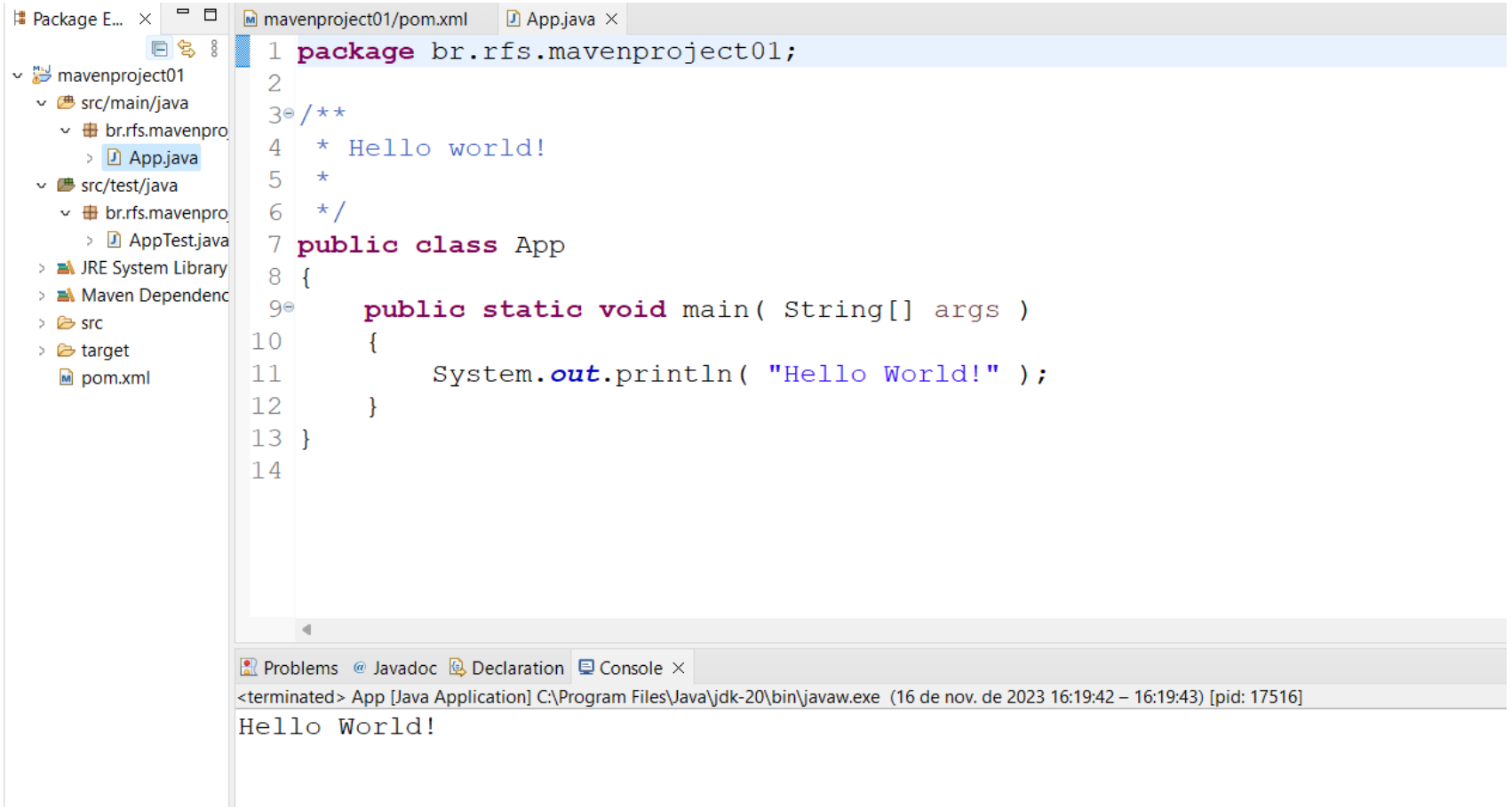
- Certifique-se de que o elemento <source> e <target> estejam configurados para pelo menos a versão 8.

```
<name>mavenproject01</name>
<!-- FIXME change it to the project's website -->
<url>http://www.example.com</url>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
</properties>
```

Maven

- Após, podemos compilar a classe gerada automaticamente.



Laboratório de desenvolvimento de software

Maven + Spring boot + MySQL

Spring boot

- O Spring Boot é um framework que visa simplificar o desenvolvimento de aplicativos Java.
- Fornece uma maneira rápida e eficiente de criar aplicativos Java robustos e altamente configuráveis com o mínimo de esforço.
- O Spring Boot se concentra em convenções sobre configurações, eliminando grande parte da complexidade associada ao desenvolvimento de aplicativos Java tradicionais.

Spring boot

- Vamos desenvolver uma API!
  - Sabem o que é uma API?

API

- Uma API (Interface de Programação de Aplicações) é um conjunto de regras e definições que permite que diferentes softwares se comuniquem entre si.
- Ela define a maneira como componentes de software devem interagir
- Objetivo:
  - Facilitar a integração entre sistemas e permitir que desenvolvedores acessem funcionalidades específicas de um software sem precisar entender completamente sua implementação interna.

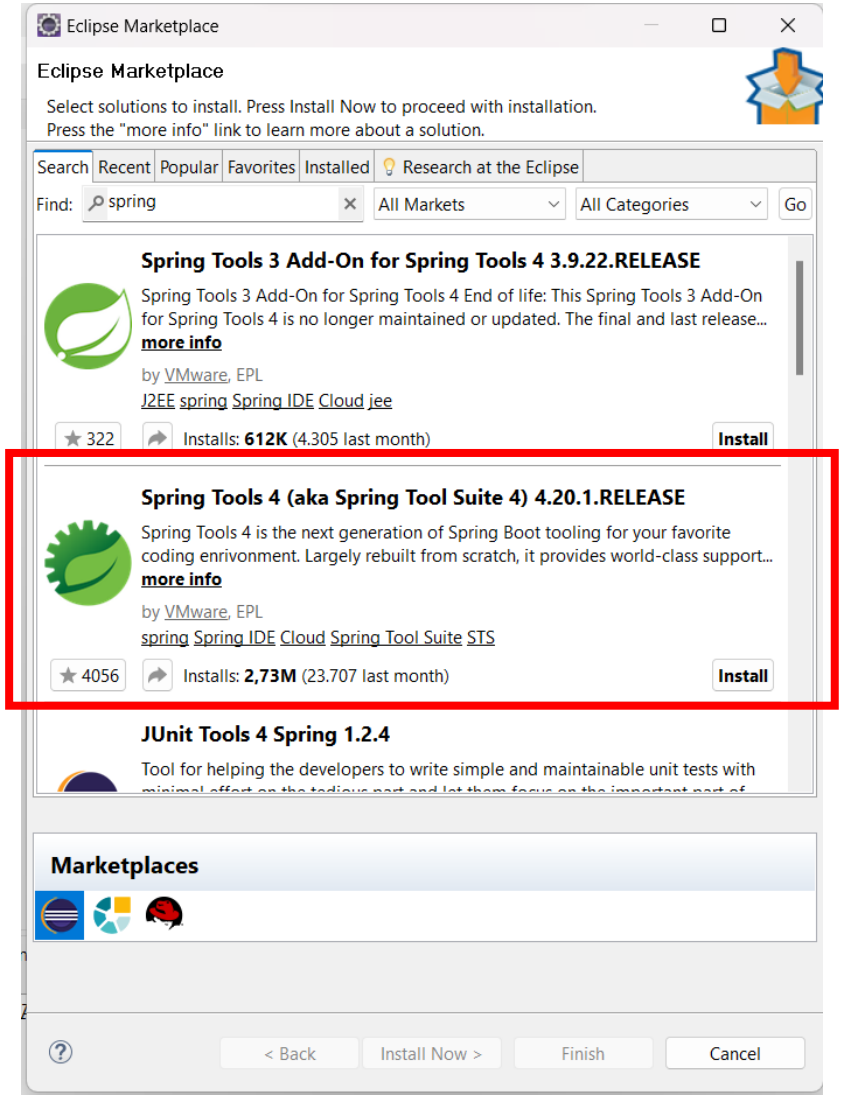
API

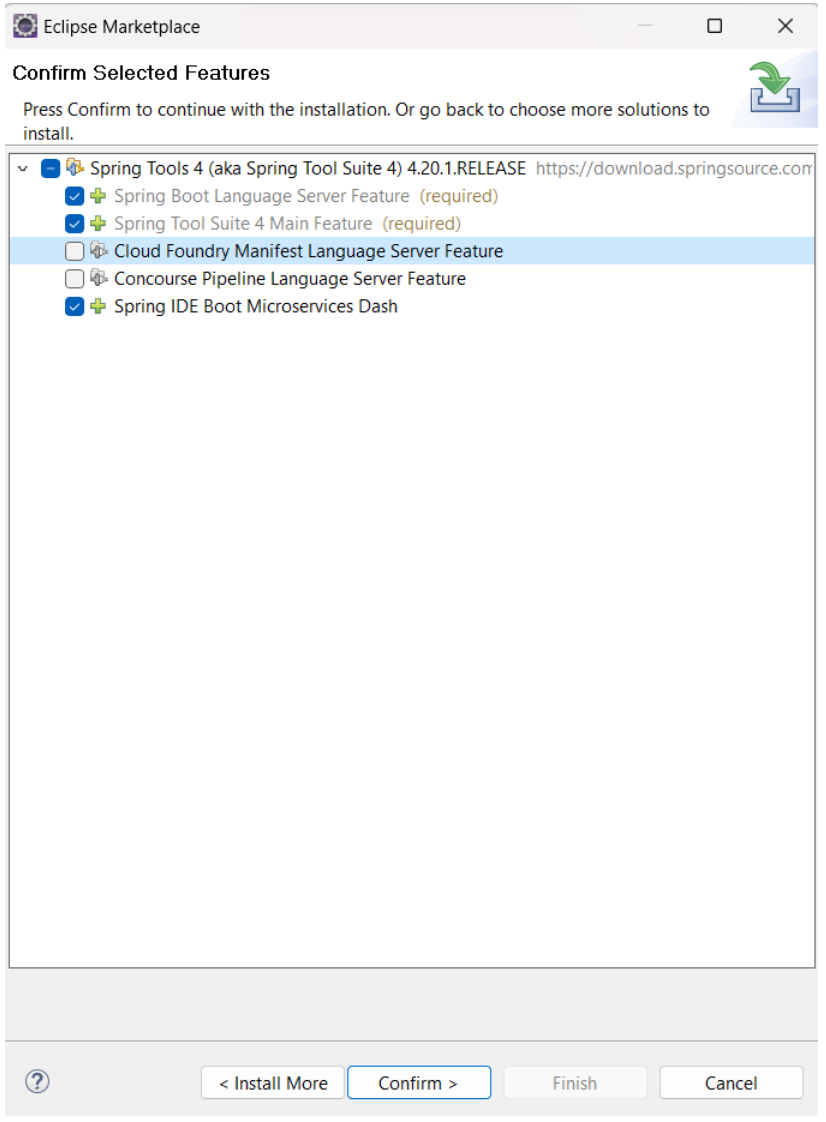
- Abstração de Funcionalidades:
  - APIs fornecem uma camada de abstração, permitindo que os desenvolvedores acessem funcionalidades específicas sem precisar entender a implementação interna.
- Padrões de Comunicação:
  - As APIs seguem padrões específicos de comunicação, como o uso de protocolos como HTTP/HTTPS e métodos (GET, POST, PUT, DELETE).
- Reutilização de Código:
  - As APIs permitem a reutilização de código e funcionalidades, acelerando o desenvolvimento.

API

- APIs Web:
  - APIs web são interfaces que utilizam os protocolos HTTP/HTTPS para comunicação. Elas são amplamente utilizadas para a construção de serviços online.
- RESTful APIs:
  - Uma categoria de APIs web baseada nos princípios da arquitetura REST (Representational State Transfer).

Instalando o Spring Boot no Eclipse Marketplace



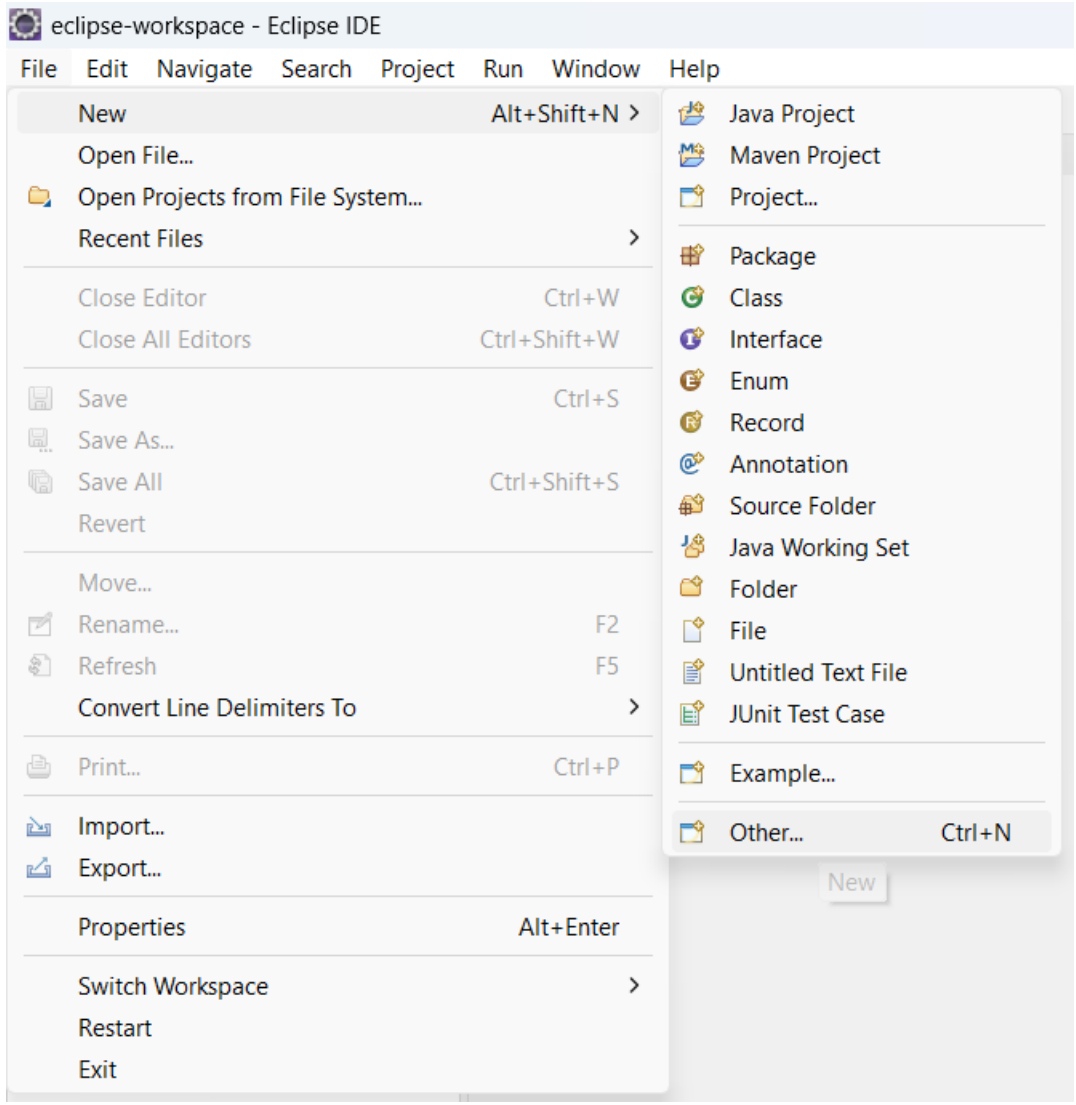


Spring boot

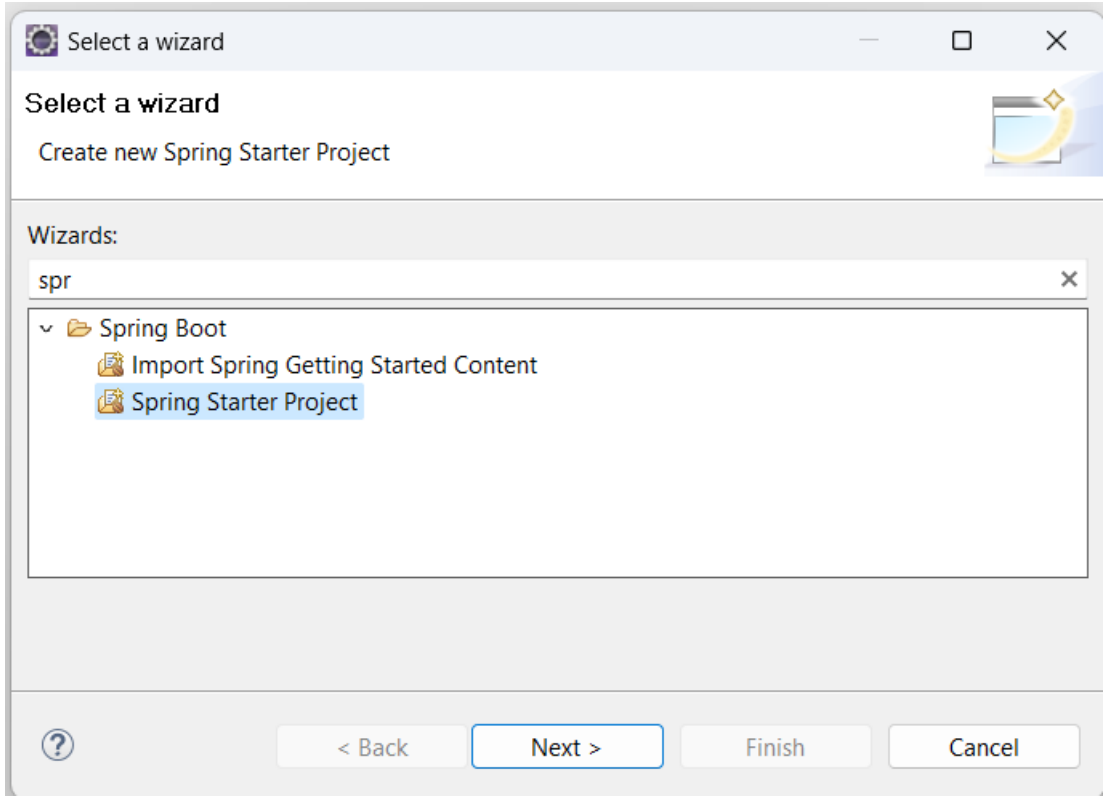
- Podemos também criar o projeto sem o Spring tools instalado
  - Acesse https://start.spring.io/
- Por lá, podemos já configurar as primeiras dependências e baixar o projeto pré-configurado.

Criando um primeiro projeto

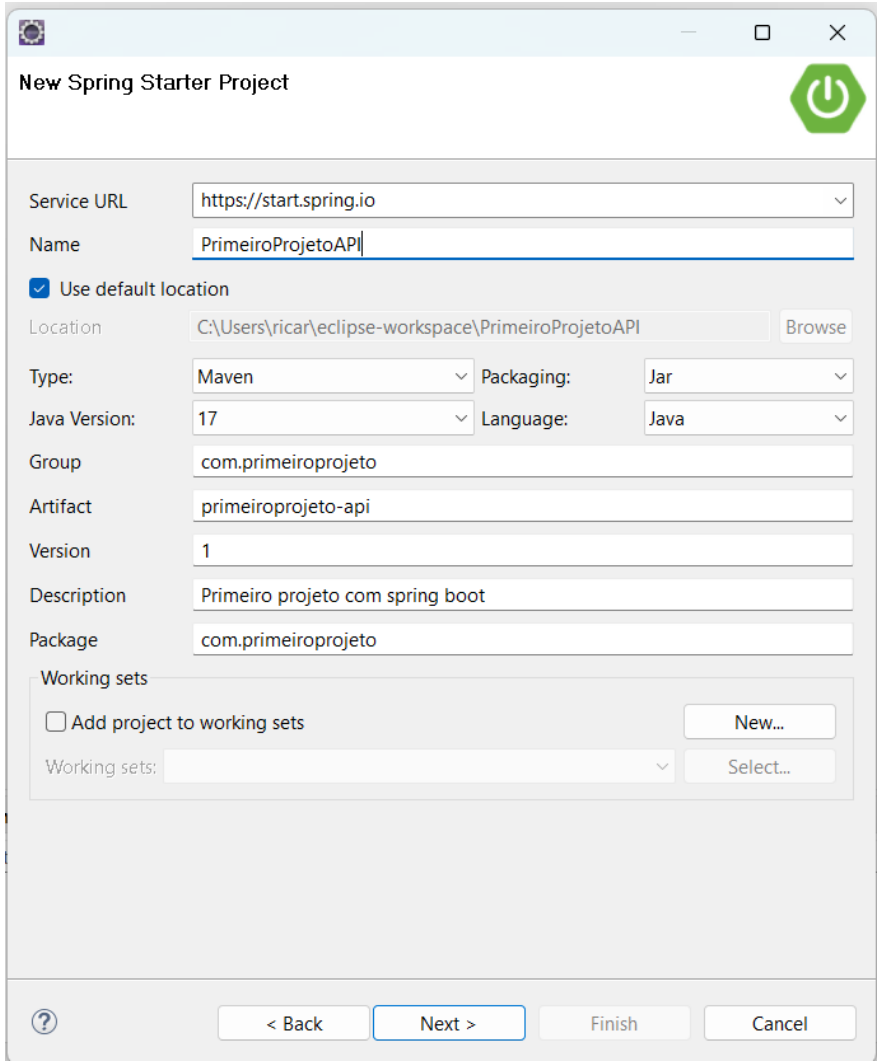
- Ao final da instalação, ele solicitará reinicialização e após isso, clique em File -> New -> Other

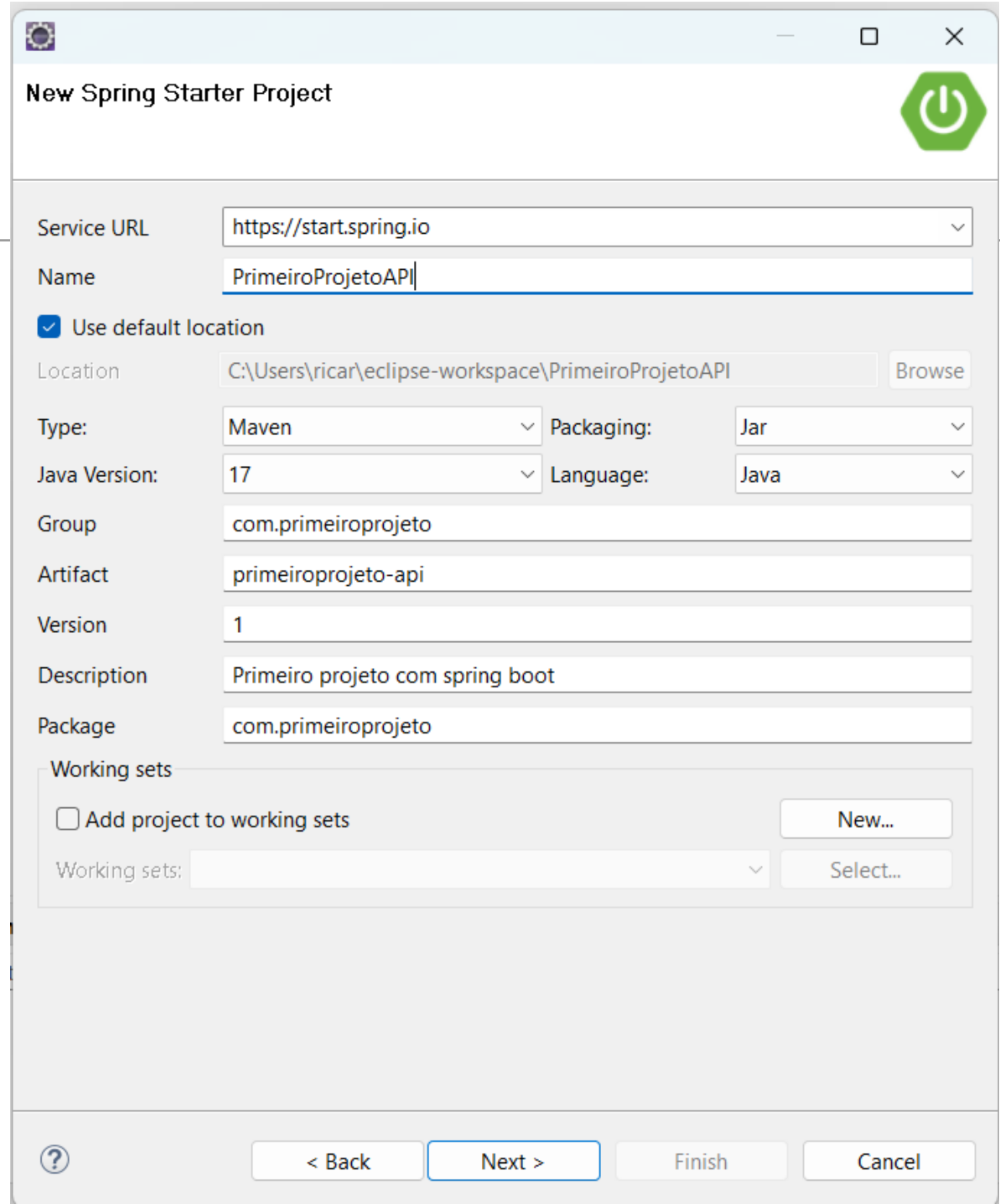


Criando um primeiro projeto

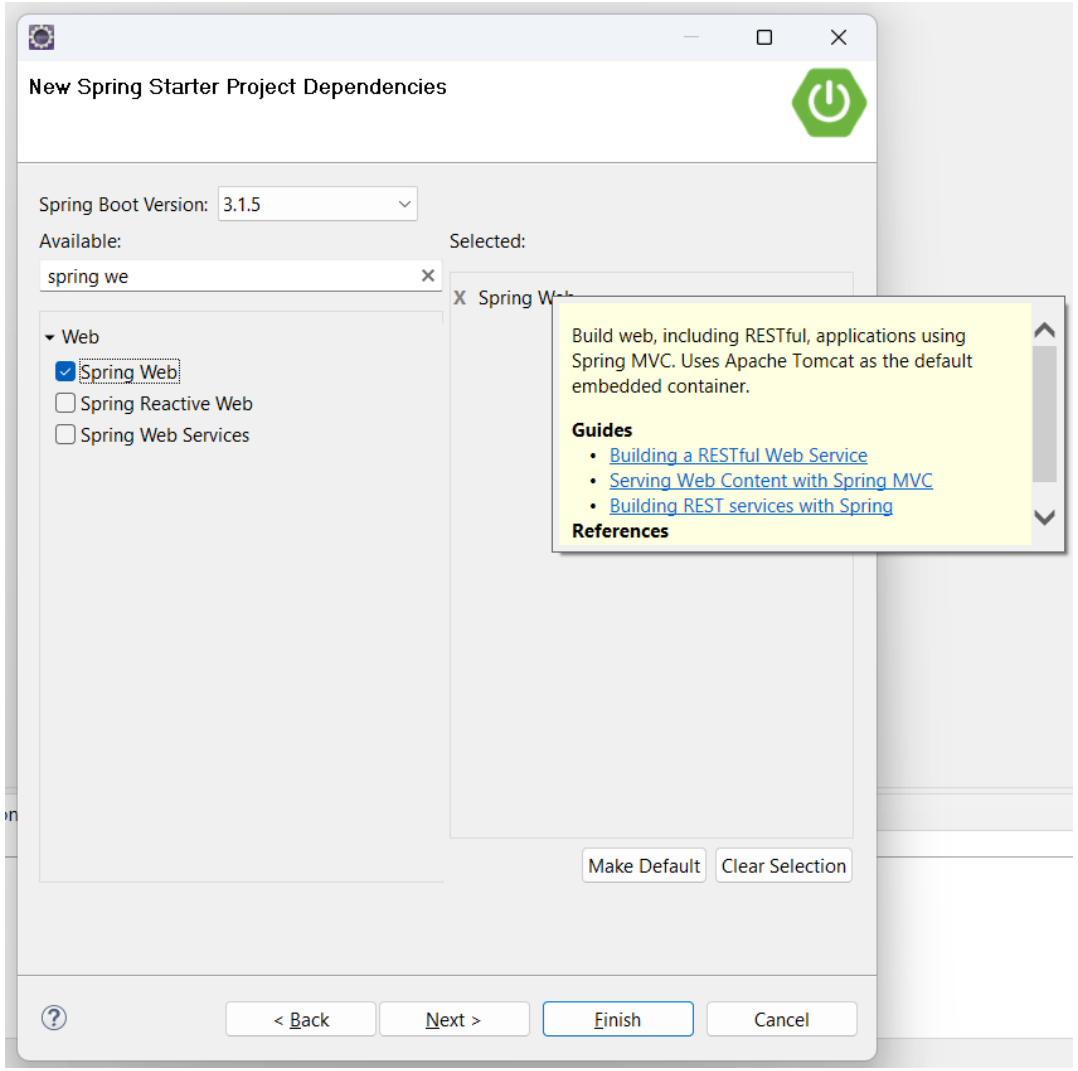


Criando um primeiro projeto

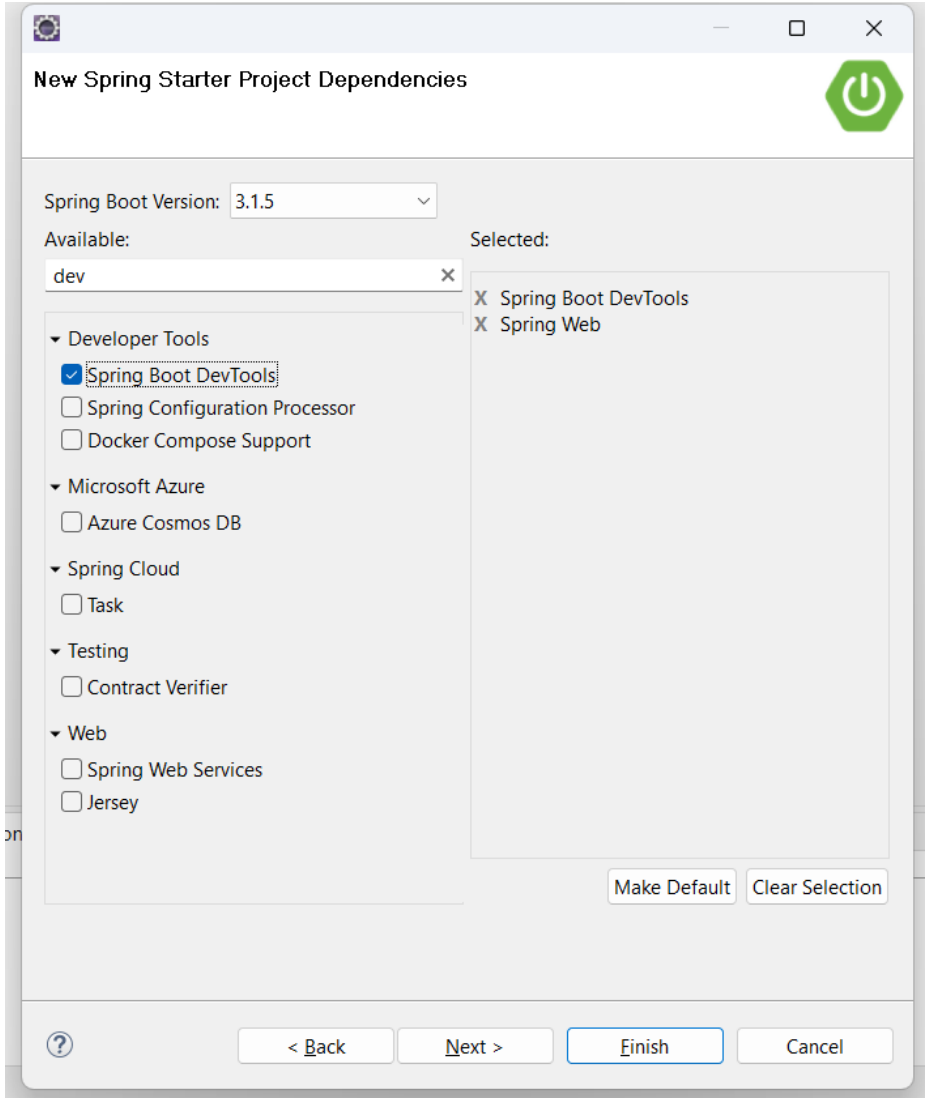



New Spring Starter Project
Service URL
https://start.spring.io
Name
PrimeiroProjetoAPI
Use default location
Location
C:\Users\ricar\eclipse-workspace\PrimeiroProjetoAPI
Browse
Type:
Maven
Packaging:
Jar
Java Version:
17
Language:
Java
Group
com.primeiroprojeto
Artifact
primeiroprojeto-api
Version
1
Description
Primeiro projeto com spring boot
Package
com.primeiroprojeto
Working sets
Add project to working sets
New...
Working sets:
Select...
< Back
Next >
Finish
Cancel

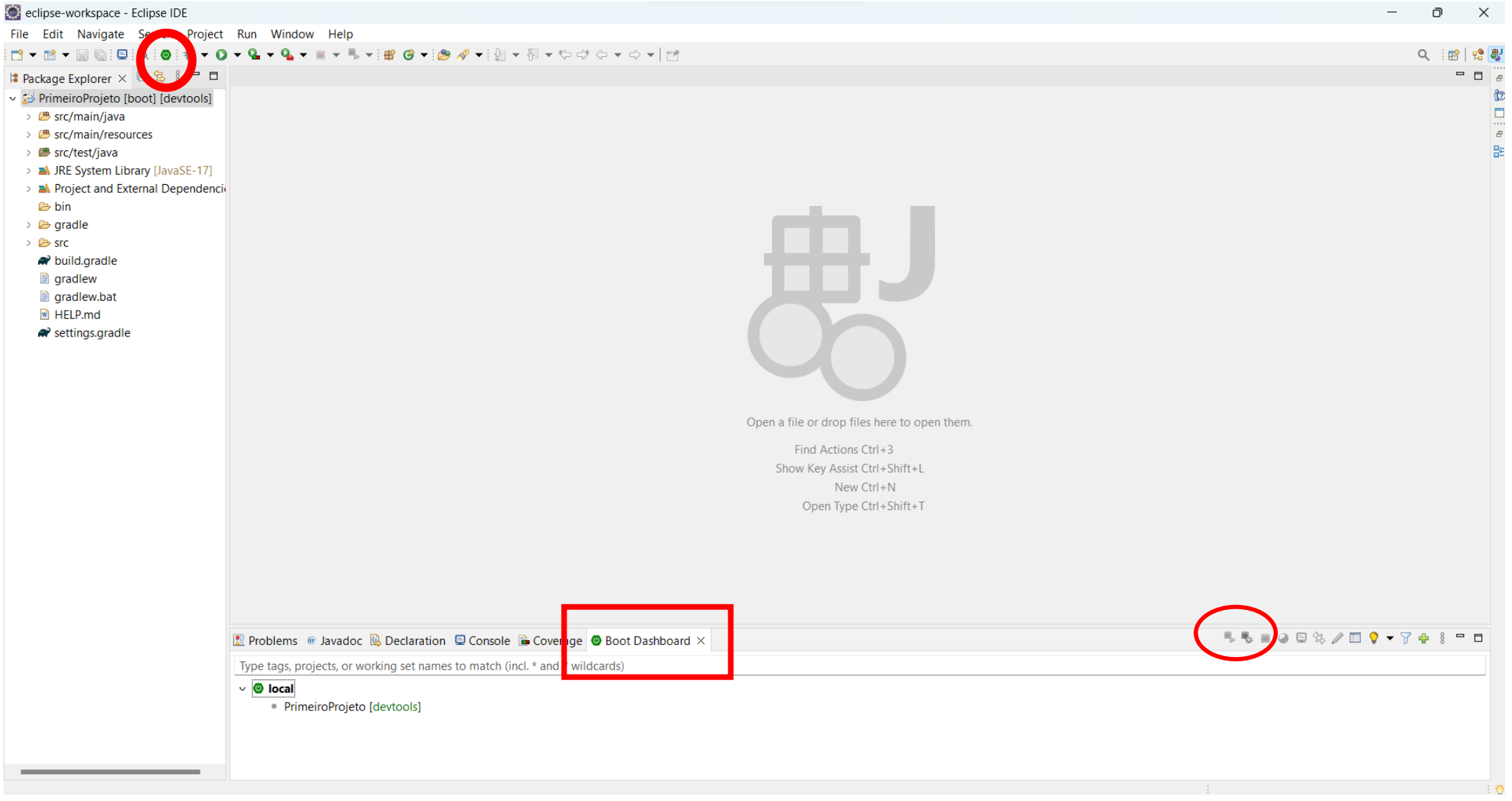
Criando um primeiro projeto



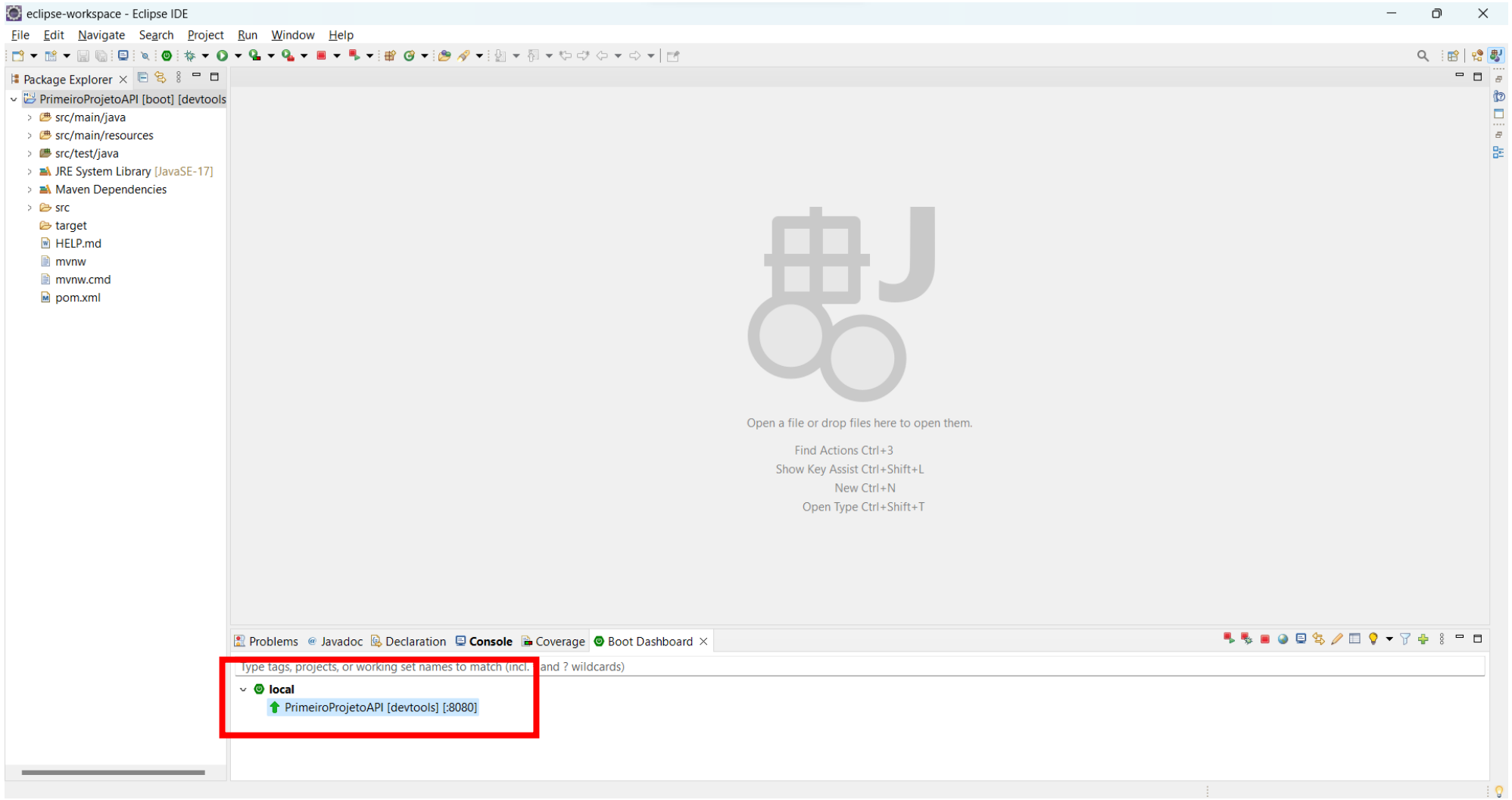
Criando um primeiro projeto



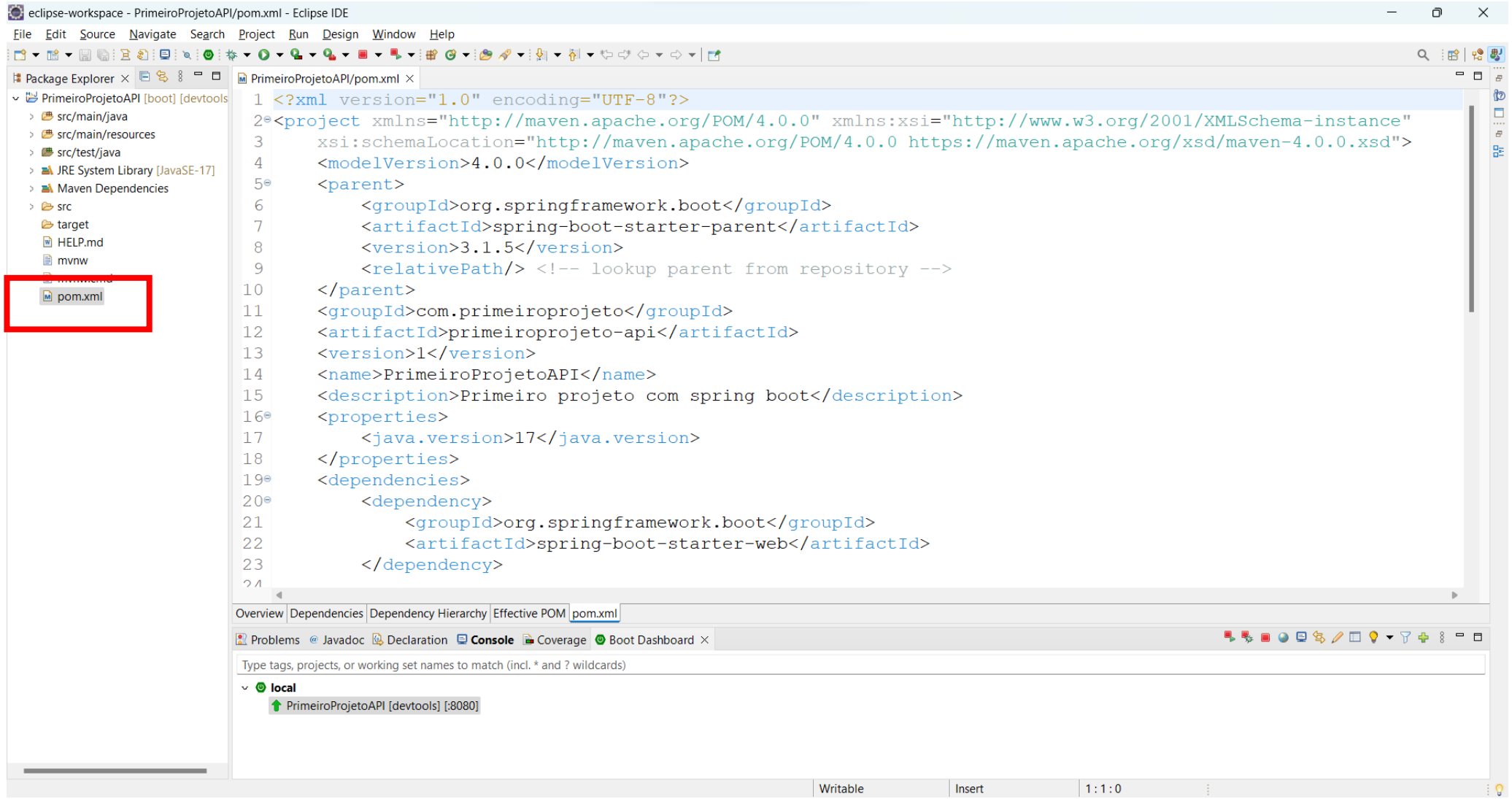
Boot Dashboard



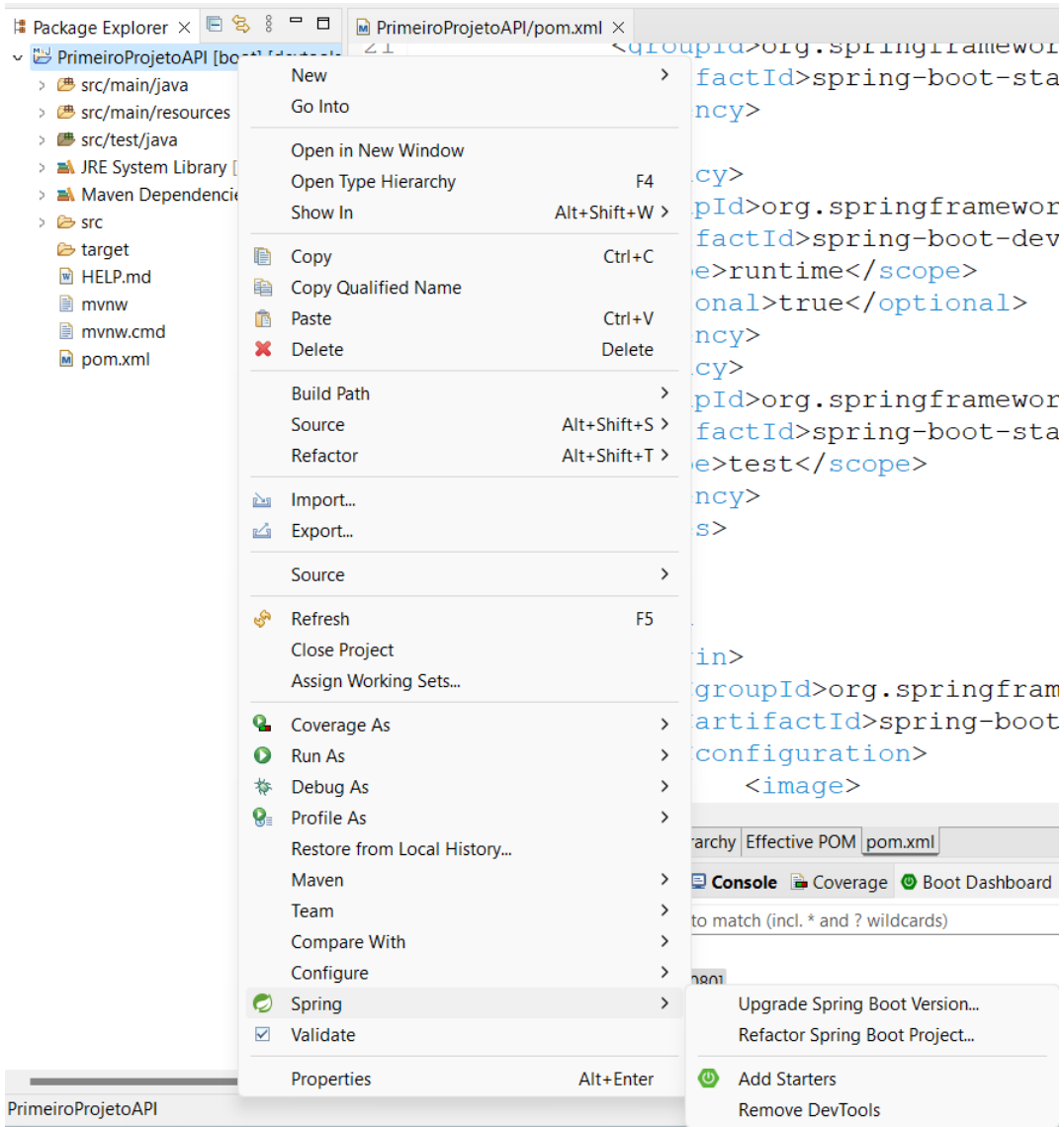
Executando o projeto



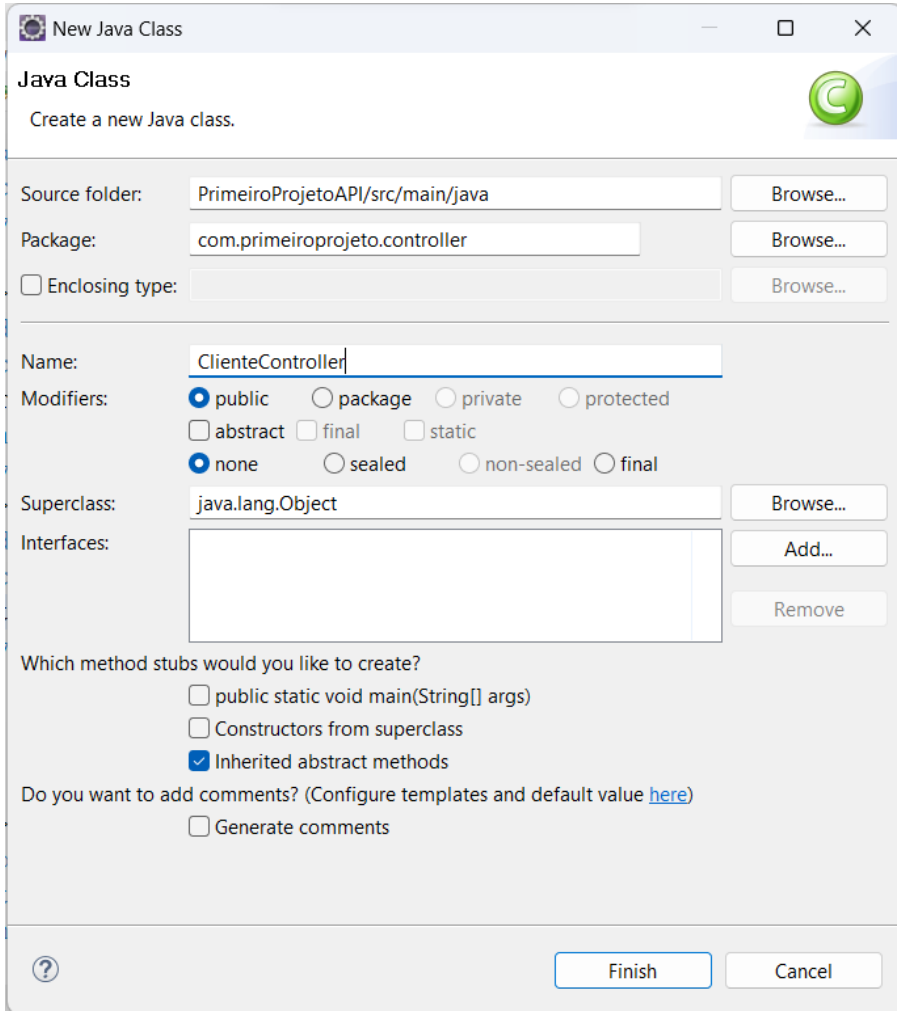
Gerenciando as dependências

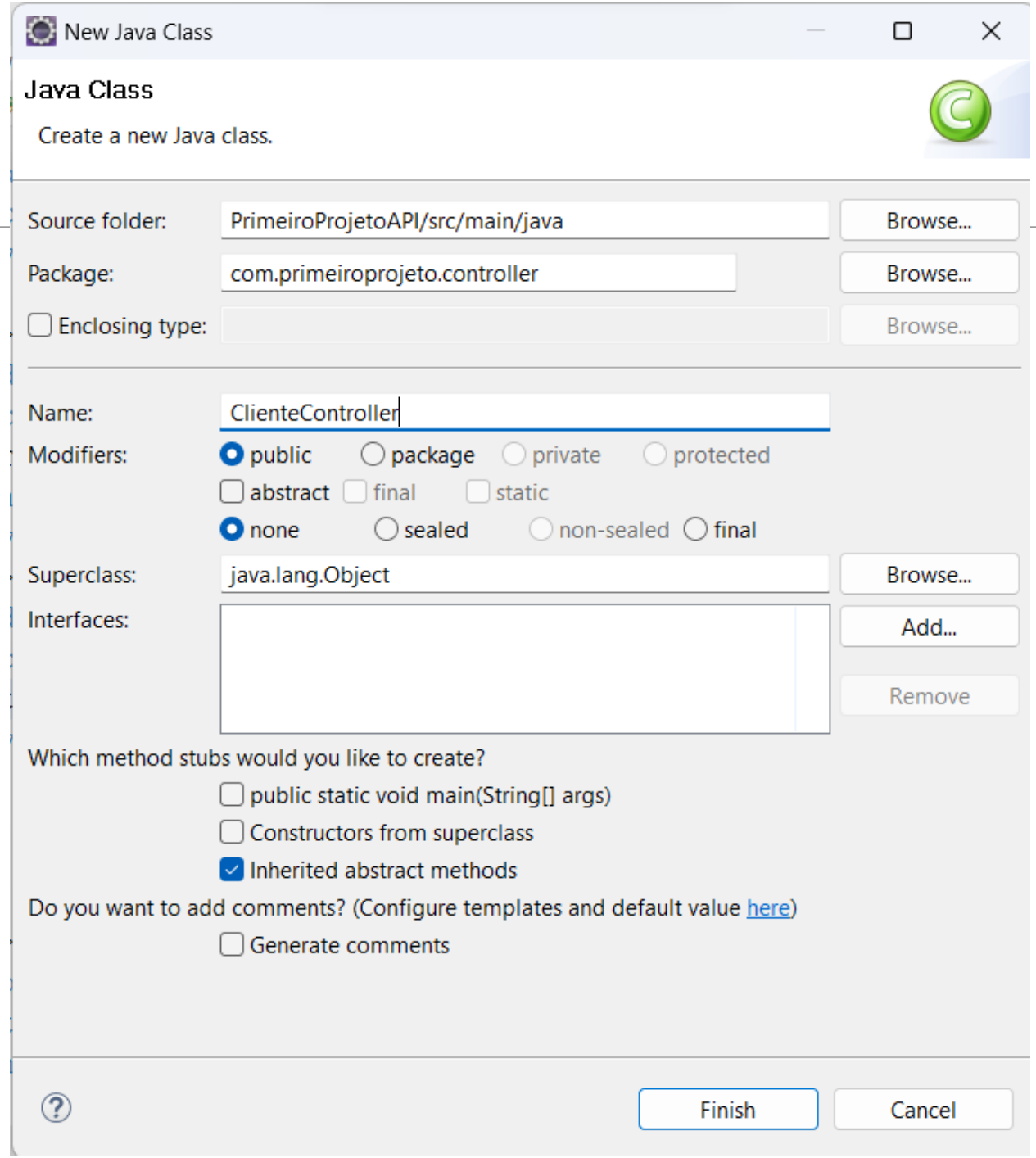


Gerenciando as dependências



Criando nossa controller




New Java Class
Java Class
Create a new Java class.
Source folder:
PrimeiroProjetoAPI/src/main/java
Browse...
Package:
com.primeiroprojeto.controller
Browse...
Enclosing type:
Browse...
Name:
ClienteController
Modifiers:
public
package
private
protected
abstract
final
static
none
sealed
non-sealed
final
Superclass:
java.lang.Object
Browse...
Interfaces:
Add...
Remove
Which method stubs would you like to create?
public static void main(String[] args)
Constructors from superclass
Inherited abstract methods
Do you want to add comments? (Configure templates and default value here)
Generate comments
Finish
Cancel

O que é Controller? Model? E View?

- MVC (Model-View-Controller) é um padrão arquitetural amplamente utilizado em projetos de software para organizar e estruturar o código-fonte de maneira modular e separada de preocupações.

- Ele separa a lógica de negócios (Model), a apresentação de dados (View) e o fluxo de controle (Controller), facilitando a manutenção, testabilidade e escalabilidade de um sistema.

O que é Controller? Model? E View?

- Model (Modelo):
  - O Model representa a lógica de negócios e os dados do aplicativo. Ele lida com a manipulação e gerenciamento dos dados, bem como as regras de negócios associadas.
  - Em um sistema MVC, o Model é frequentemente representado por classes ou componentes que encapsulam a lógica de negócios e interagem com o banco de dados, se aplicável.
  - O Model notifica a View e o Controller sobre mudanças nos dados, permitindo que eles atualizem e reajam adequadamente.

O que é Controller? Model? E View?

- View (Visualização):
  - A View é responsável por apresentar os dados ao usuário e interagir com ele. Ela exibe a interface do usuário e recebe entradas do usuário.
  - A View não contém lógica de negócios significativa; em vez disso, ela reflete o estado atual do Model e exibe as informações de maneira compreensível para o usuário.
  - No contexto da web, a View geralmente consiste em páginas HTML, enquanto em aplicativos de desktop, ela pode ser composta por janelas, painéis e outros componentes gráficos.

O que é Controller? Model? E View?

- Controller (Controlador):
  - O Controller gerencia as interações do usuário e atua como um intermediário entre a View e o Model. Ele recebe entradas do usuário da View, processa essas entradas e atualiza o Model de acordo.
  - O Controller também recebe notificações do Model sobre mudanças nos dados e atualiza a View para refletir essas alterações.
  - Em aplicações web, o Controller geralmente é responsável por interpretar as solicitações HTTP, invocar a lógica de negócios correspondente e retornar a resposta apropriada.

ClienteController

```java
package com.primeiroprojeto.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/clientes")
public class ClienteController {

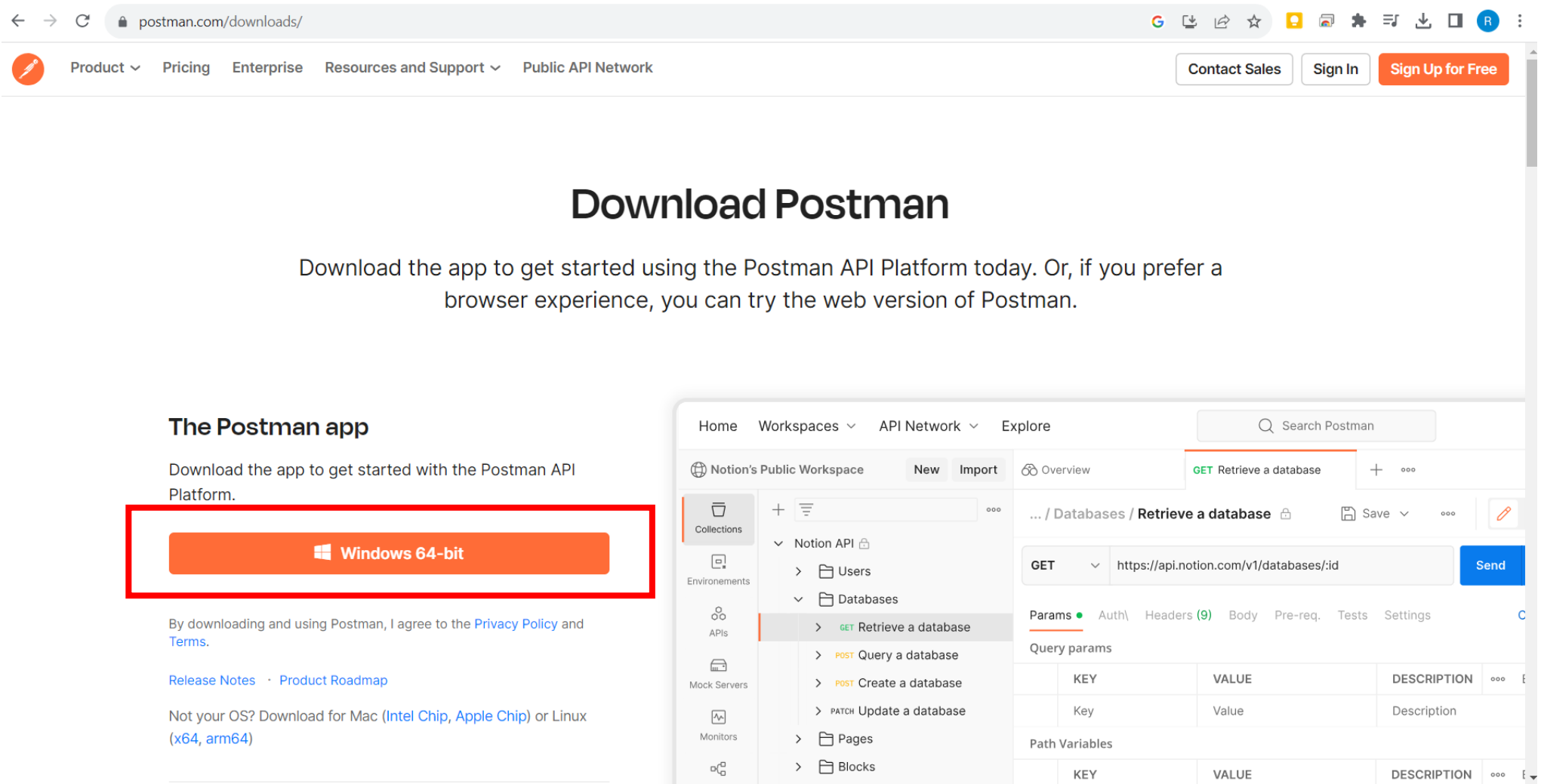
    @GetMapping
    public String hello() {
        return "Olá, mundo!";
    }

}
```

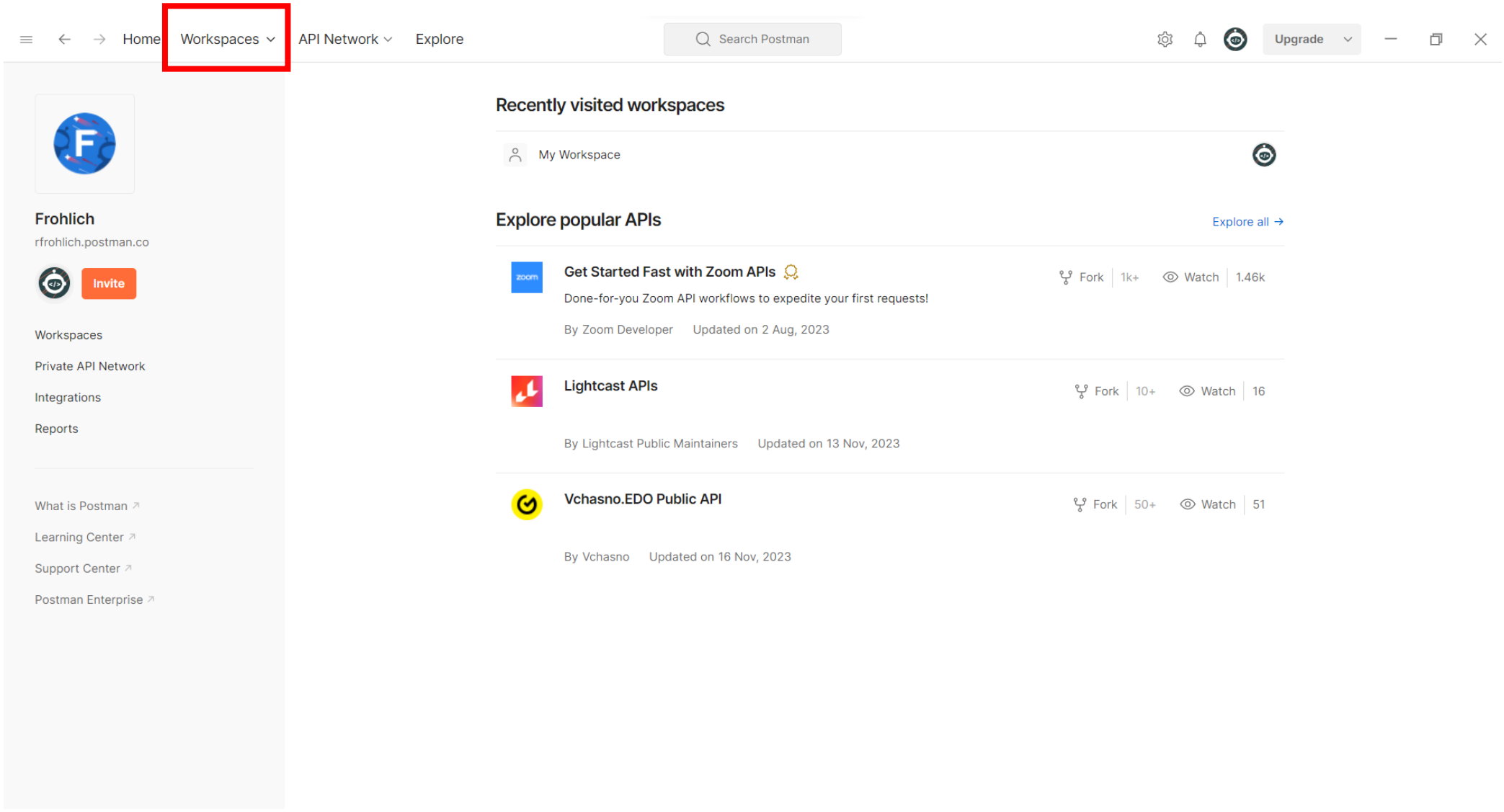
Testando o acesso

- Tente acessar http://localhost:8080/clientes

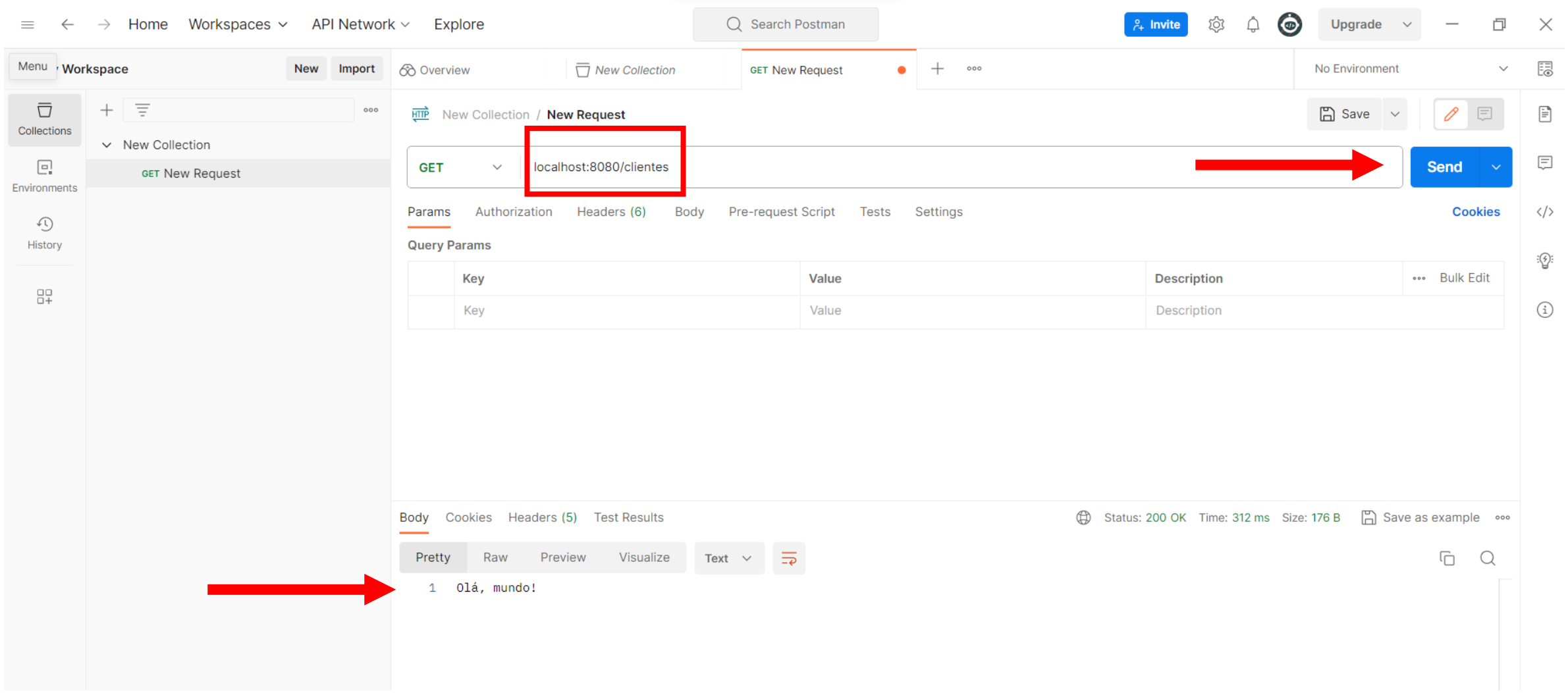



postman.com/downloads/
Product
Pricing
Enterprise
Resources and Support
Public API Network
Contact Sales
Sign In
Sign Up for Free
Download Postman
Download the app to get started using the Postman API Platform today. Or, if you prefer a browser experience, you can try the web version of Postman.
The Postman app
Download the app to get started with the Postman API Platform.
Windows 64-bit
By downloading and using Postman, I agree to the Privacy Policy and Terms.
Release Notes · Product Roadmap
Not your OS? Download for Mac (Intel Chip, Apple Chip) or Linux (x64, arm64)
Home
Workspaces
API Network
Explore
Search Postman
Notion's Public Workspace
New
Import
Overview
GET Retrieve a database
Collections
Environements
APIs
Mock Servers
Monitors
Notion API
Users
Databases
GET Retrieve a database
POST Query a database
POST Create a database
PATCH Update a database
Pages
Blocks
... / Databases / Retrieve a database
Save
GET
https://api.notion.com/v1/databases/:id
Send
Params
Auth\
Headers (9)
Body
Pre-req.
Tests
Settings
Query params
KEY
VALUE
DESCRIPTION
Key
Value
Description
Path Variables
KEY
VALUE
DESCRIPTION

Postman


Home
Workspaces
API Network
Explore
Search Postman
Upgrade
Frohlich
rfrohlich.postman.co
Invite
Workspaces
Private API Network
Integrations
Reports
What is Postman
Learning Center
Support Center
Postman Enterprise
Recently visited workspaces
My Workspace
Explore popular APIs
Explore all →
Get Started Fast with Zoom APIs
Done-for-you Zoom API workflows to expedite your first requests!
By Zoom Developer
Updated on 2 Aug, 2023
Fork 1k+
Watch 1.46k
Lightcast APIs
By Lightcast Public Maintainers
Updated on 13 Nov, 2023
Fork 10+
Watch 16
Vchasno.EDO Public API
By Vchasno
Updated on 16 Nov, 2023
Fork 50+
Watch 51

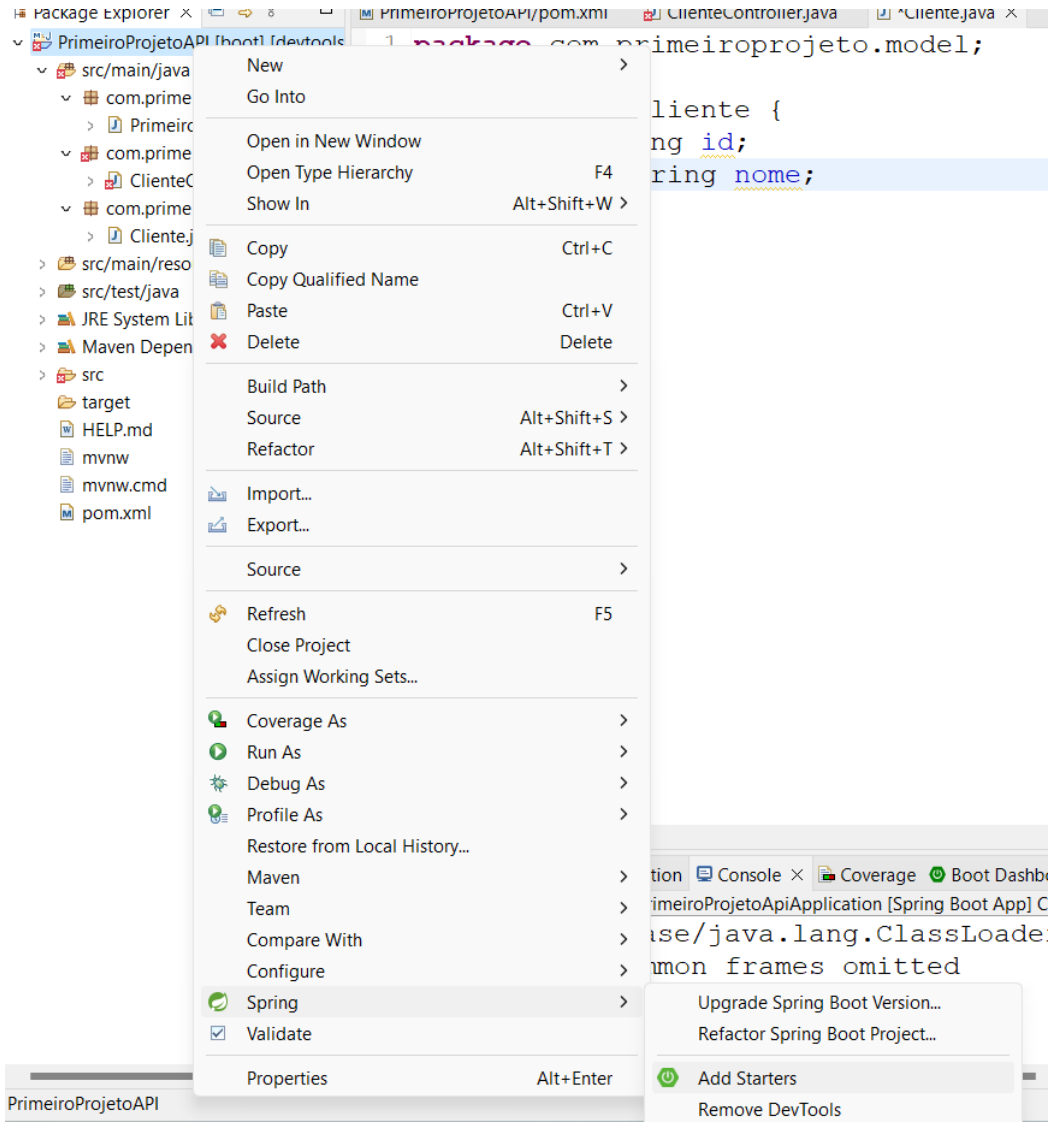
Postman

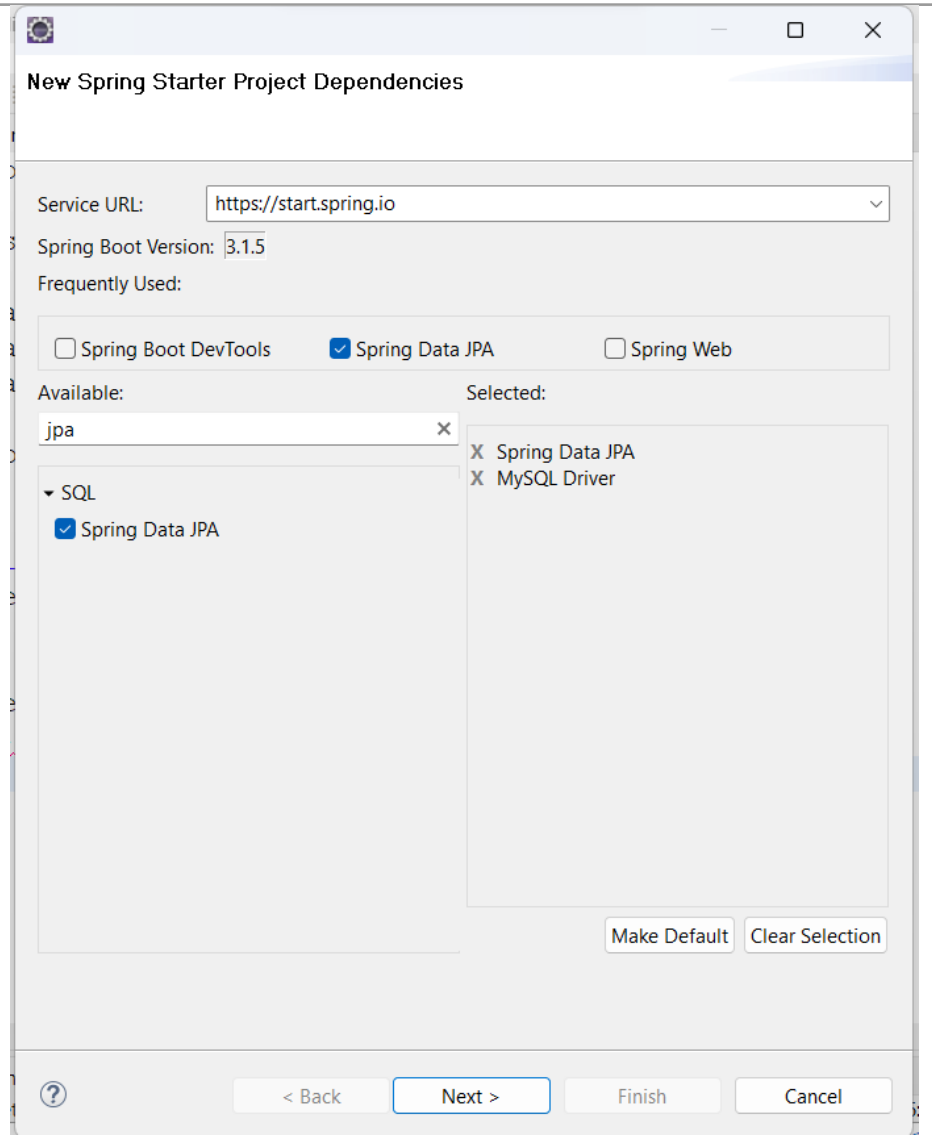

Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Menu
Workspace
New
Import
Overview
New Collection
GET New Request
No Environment
Collections
Environments
History
New Collection
GET New Request
New Collection / New Request
Save
GET
localhost:8080/clientes
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 312 ms
Size: 176 B
Save as example
Pretty
Raw
Preview
Visualize
Text
1 Olá, mundo!

O que nos falta?

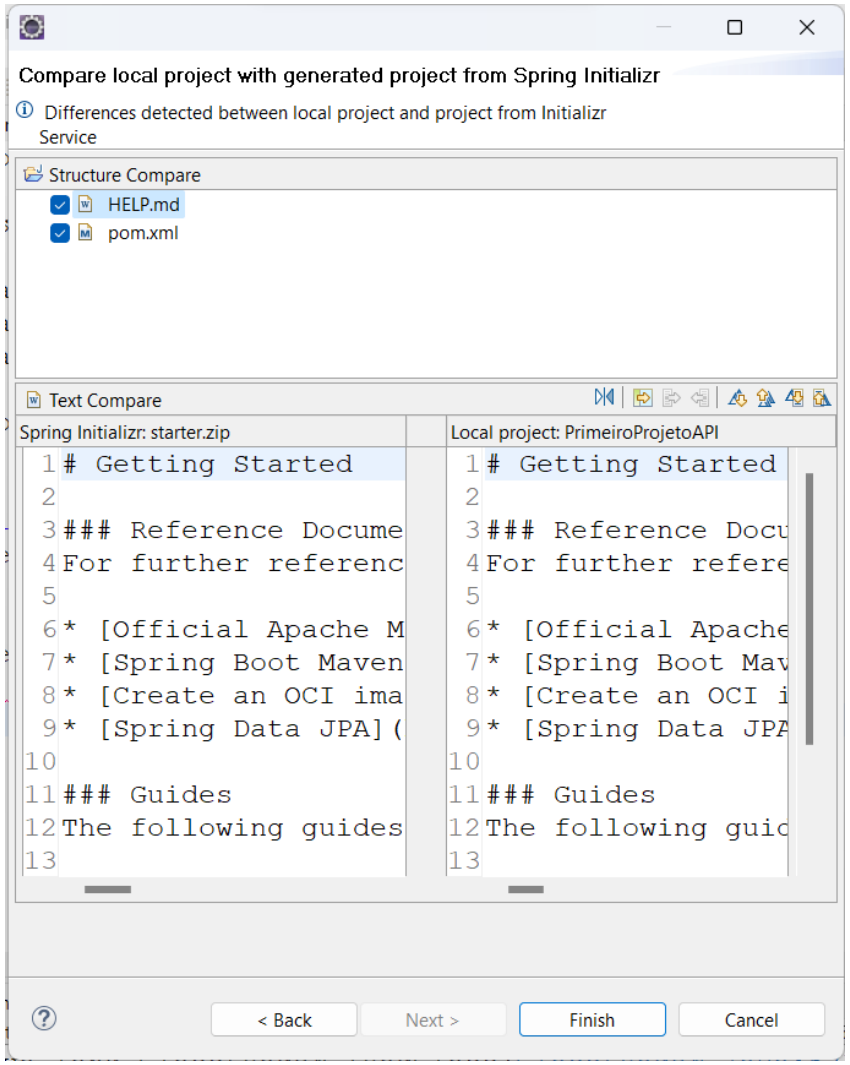
- Precisamos conectar ao banco para armazenar dados...

Adicionando dependências





Adicionando dependências



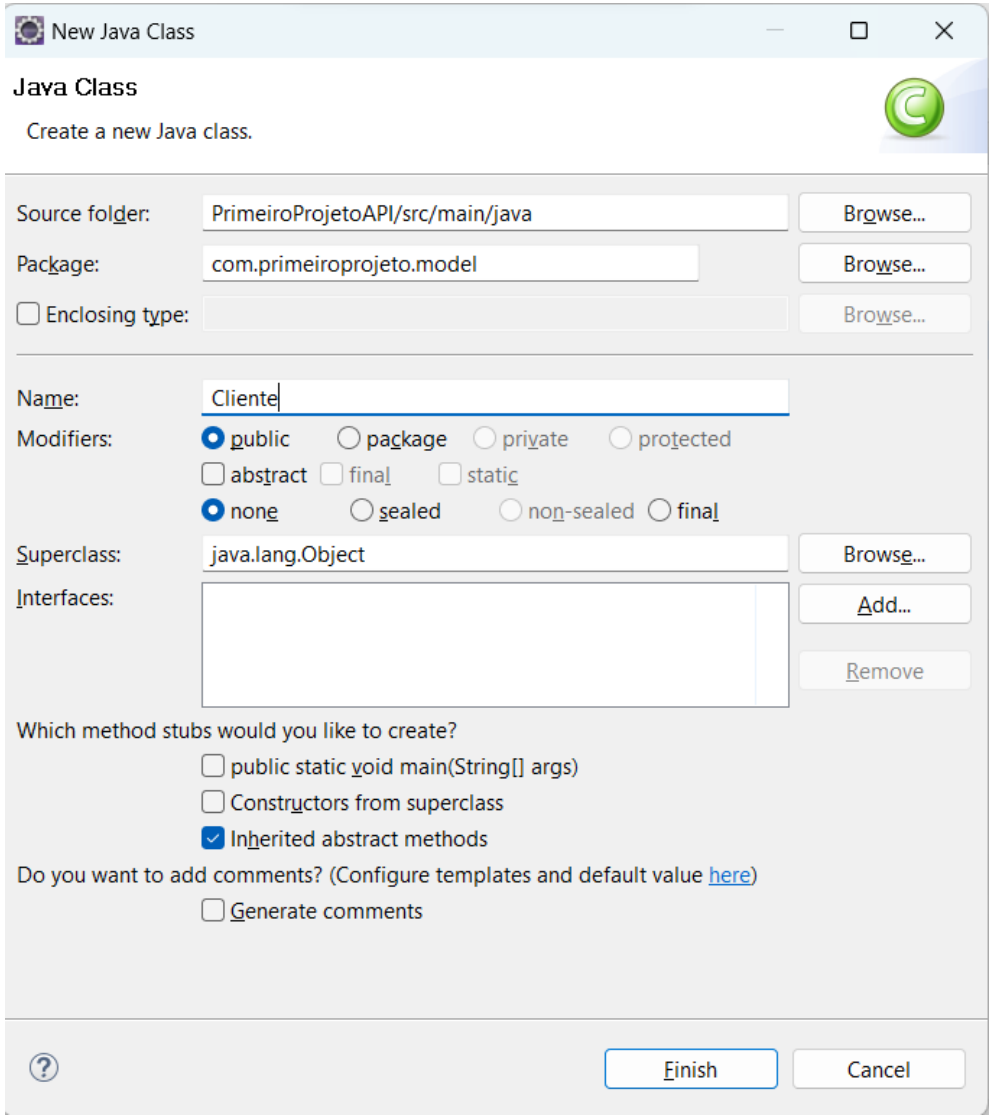
Banco de dados

- Agora no projeto, procure o arquivo “application.properties”

PrimeiroProjetoAPI/pom.xml | ClienteController.java | ClienteRepository.java | application.properties | PrimeiroProjetoAPI/pom.xml | Cliente.java

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/bancoClientes?createDatabaseIfNotExist=true
2 spring.datasource.username=root
3 spring.datasource.password=1234
4 spring.jpa.hibernate.ddl-auto=update
```

Agora, vamos criar a classe Cliente



Classe Cliente

```java
import java.util.Objects;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Cliente {
    @Id
    @GeneratedValue (strategy = GenerationType.IDENTITY)
    private Long id;

    @Column (nullable = false)
    private String nome;

    public Long getId() {
        return id;
    }

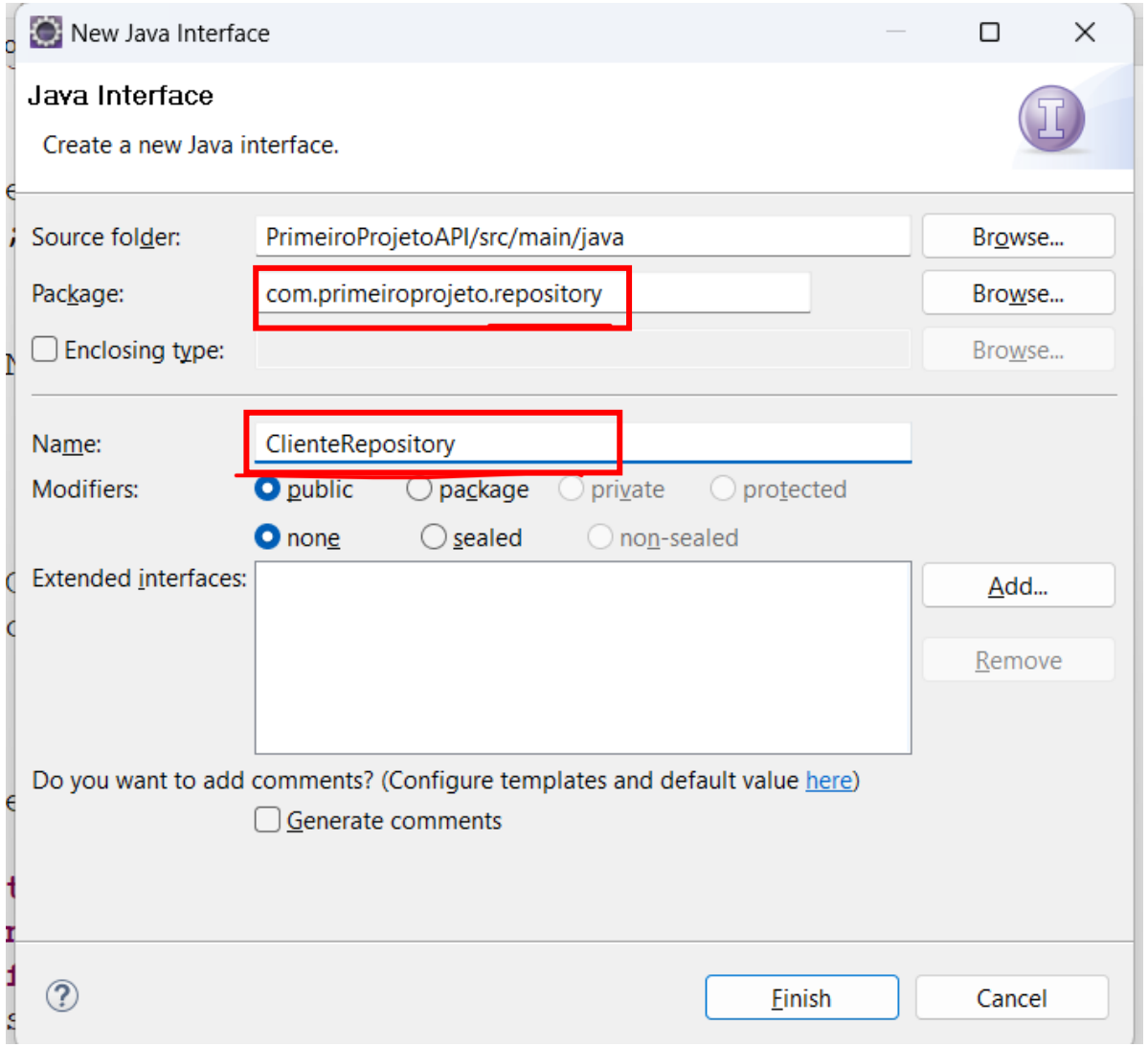
    public void setId(Long id) {
        this.id = id;
    }

```

Classe Cliente

```
28      public String getNome() {
29          return nome;
30      }
31
32      public void setNome(String nome) {
33          this.nome = nome;
34      }
35
36      @Override
37      public int hashCode() {
38          return Objects.hash(id, nome);
39      }
40
41      @Override
42      public boolean equals(Object obj) {
43          if (this == obj)
44              return true;
45          if (obj == null)
46              return false;
47          if (getClass() != obj.getClass())
48              return false;
49          Cliente other = (Cliente) obj;
50          return Objects.equals(id, other.id) && Objects.equals(nome, other.nome);
51      }
52  }
```

Interface para o repositório de dados



Interface para o repositório de dados

```java
package com.primeiroprojeto.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.primeiroprojeto.model.Cliente;

@Repository
public interface ClienteRepository extends JpaRepository<Cliente, Long>{

}
```

Classe ClienteController

```
20 @RestController
21 @RequestMapping("/clientes")
22 public class ClienteController {
23
24     private final ClienteRepository clienteRepository;
25
26     public ClienteController(ClienteRepository clienteRepository) {
27         super();
28         this.clienteRepository = clienteRepository;
29     }
30
31     @GetMapping
32     public List<Cliente> listar() {
33         return clienteRepository.findAll();
34     }
```

Banco de dados

- Crie um banco de dados chamado “bancoclientes” no mysql.
    - create database bancoclientes;

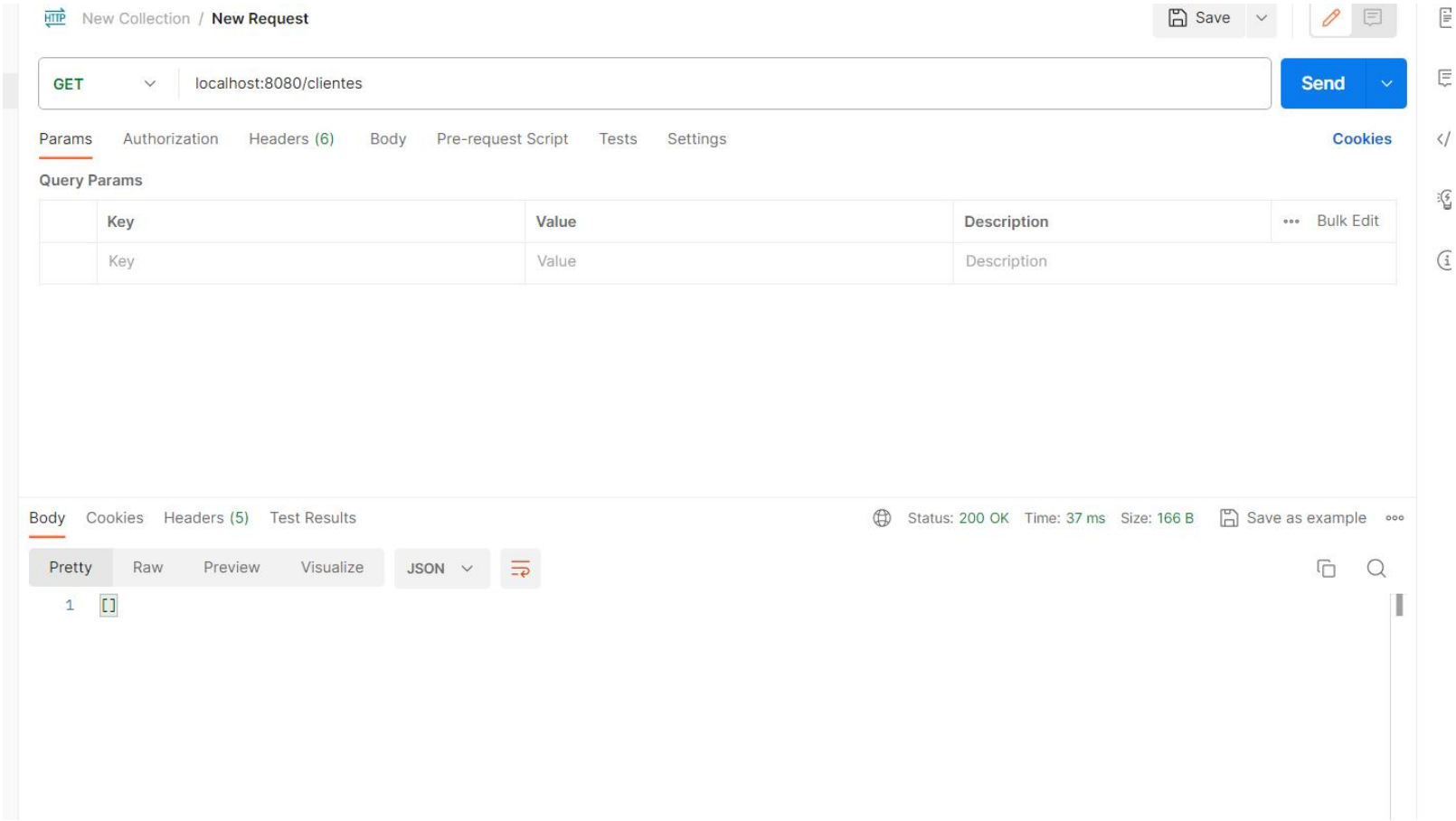
No pom.xml

- Verifique se está dependência está lá

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.23</version>
</dependency>
```

Postman

- No postman, já temos o retorno vazio do banco de dados.

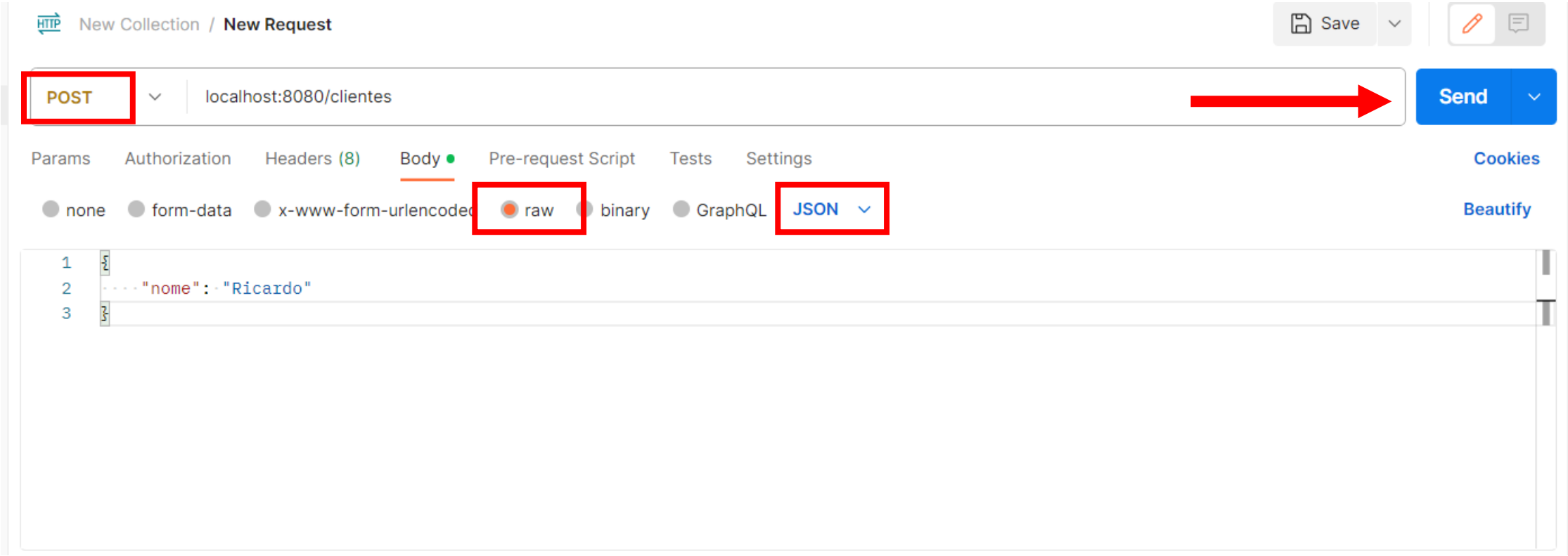


Método POST

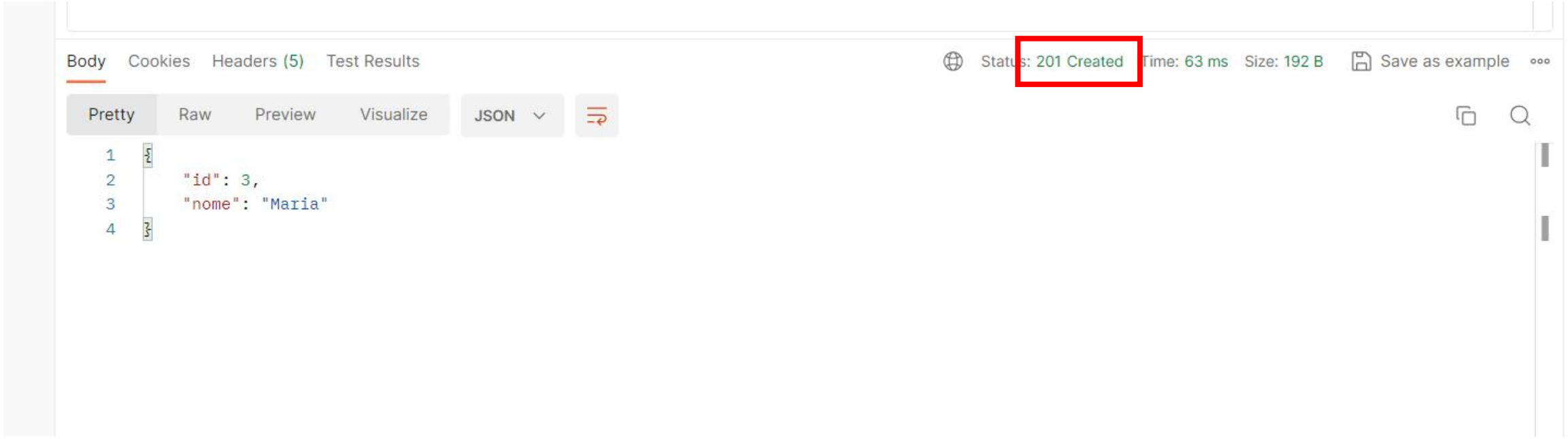
- Agora, vamos cadastrar no sistema, para isso, precisamos mapear o Post na nossa Controller

```
20 @RestController
21 @RequestMapping("/clientes")
22 public class ClienteController {
23
24     private final ClienteRepository clienteRepository;
25
26     public ClienteController(ClienteRepository clienteRepository) {
27         super();
28         this.clienteRepository = clienteRepository;
29     }
30
31     @GetMapping
32     public List<Cliente> listar() {
33         return clienteRepository.findAll();
34     }
35
36     @PostMapping
37     @ResponseStatus(HttpStatus.CREATED)
38     public Cliente adicionar (@RequestBody Cliente cliente) {
39         return clienteRepository.save(cliente);
40     }
41 }
```

Postman


New Collection / New Request
Save
POST
localhost:8080/clientes
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
1 {
2 "nome": "Ricardo"
3 }

Postman

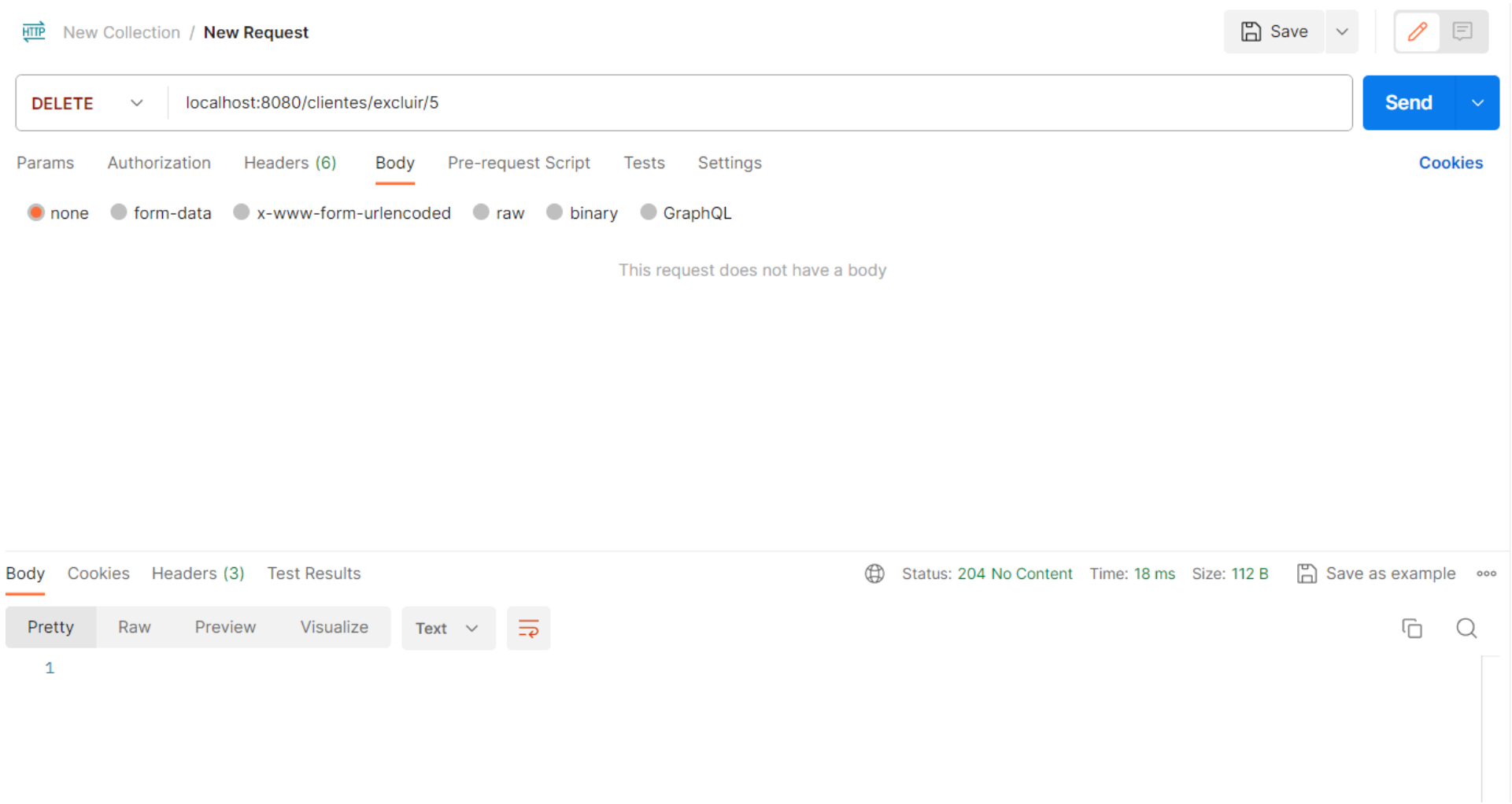

Body
Cookies
Headers (5)
Test Results
Status: 201 Created
Time: 63 ms
Size: 192 B
Save as example
Pretty
Raw
Preview
Visualize
JSON
{
"id": 3,
"nome": "Maria"
}

Criar o delete na API

- Na ClienteController, adicione o método:

```
37    @DeleteMapping("/excluir/{id}")
38    public ResponseEntity<Void> deletarCliente(@PathVariable Long id) {
39
40        if (!clienteRepository.existsById(id)) {
41            return ResponseEntity.notFound().build();
42        }
43
44        // Remove o cliente do banco de dados
45        clienteRepository.deleteById(id);
46
47        return ResponseEntity.noContent().build();
48    }
```

No postman


New Collection / New Request
Save
DELETE
localhost:8080/clientes/excluir/5
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
This request does not have a body
Body
Cookies
Headers (3)
Test Results
Status: 204 No Content
Time: 18 ms
Size: 112 B
Save as example
Pretty
Raw
Preview
Visualize
Text
1

E agora, o editar:

```
@PutMapping("/{id}")
public ResponseEntity<Cliente> atualizarCliente(@PathVariable Long id, @RequestBody Cliente clienteAtualizado) {

    if (!clienteRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    Cliente clienteExistente = clienteRepository.findById(id).orElse(null);

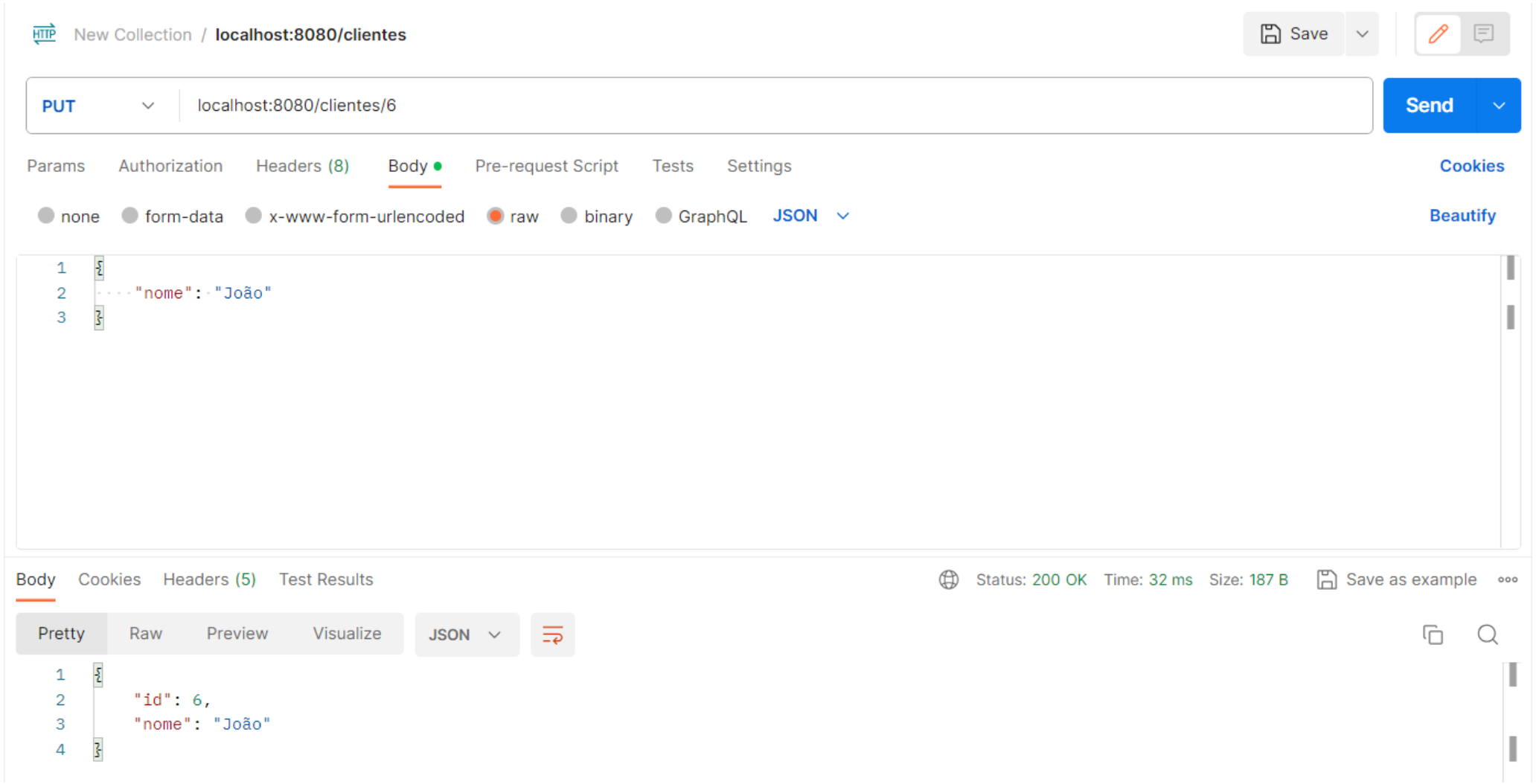
    if (clienteExistente != null) {

        clienteExistente.setNome(clienteAtualizado.getNome());


        Cliente clienteAtualizadoNoBanco = clienteRepository.save(clienteExistente);

        return ResponseEntity.ok(clienteAtualizadoNoBanco);
    } else {
        return ResponseEntity.notFound().build();
    }
}
```

No postman


New Collection / localhost:8080/clientes
Save
PUT
localhost:8080/clientes/6
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
1 {
2 "nome": "João"
3 }
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 32 ms
Size: 187 B
Save as example
Pretty
Raw
Preview
Visualize
JSON
1 {
2 "id": 6,
3 "nome": "João"
4 }